UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA

Bacharelado em Engenharia de Software

Disciplina: Domínios de Software 2026/2
Docente: Adriana Silveira Souza

Tarefa: APS – Projeto de software Web para autenticação de usuários

Autores
Ana Luísa Pereira dos Santos
Davi Augusto Ferreira de Carvalho
Isadora Yasmim da Silva

Goiânia, outubro de 2026

Autoria e revisão dos itens de informação

Item de informação	Autoria	Revisão
1.1 – Visão geral e navegador	Ana Luísa Pereira dos Santos	Davi Augusto Ferreira de Carvalho
1.2 – Comunicação em rede	Isadora Yasmim da Silva	Ana Luísa Pereira dos Santos
1.3 – Servidor web, servidor de aplicação e SGBD	Davi Augusto Ferreira de Carvalho	Isadora Yasmim da Silva
2.1 – Escopo e requisitos	Isadora Yasmim da Silva	Ana Luísa Pereira dos Santos
2.2 – Arquitetura	Ana Luísa Pereira dos Santos	Davi Augusto Ferreira de Carvalho
2.3 – Desenho detalhado de componentes	Davi Augusto Ferreira de Carvalho	Isadora Yasmim da Silva

Sumário

Seção 1 – Ambiente operacional de execução de uma aplicação Web.....4
Item 1.1 – Visão geral do ambiente e lado cliente (navegador).....4
Figura 1 – Diagrama de implantação do ambiente Web (portas padrão, configuráveis; FIELDING et al., 2022; SPRING, s.d.; POSTGRESQL, s.d.)..... 4
Item 1.2 – Comunicação em rede: DNS, TCP, TLS e HTTP.....7
Figura 3 – Diagrama de sequência da comunicação em rede na autenticação..... 9
Tabela 2 – Protocolos de rede: invocação, processamento e resultado.....10
Referências.....12
Apêndice I-Códigos PlantUML.....13
Figura 3 – código PlantUML..... 13

Seção 1 – Ambiente operacional de execução de uma aplicação Web

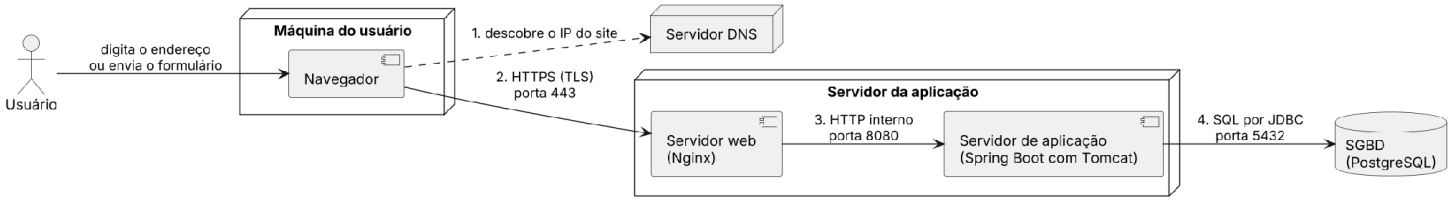
Item 1.1 – Visão geral do ambiente e lado cliente (navegador)

Autoria	Ana Luísa Pereira dos Santos
Revisão	Davi Augusto Ferreira de Carvalho
Referências usadas	FIELDING et al. (2022); MDN (2026a, 2026b, 2026c, 2026d, 2026e); POSTGRESQL (s.d.); SPRING (s.d.); Anthropic (2026), uso de IA registrado no Apêndice VI.
Itens relacionados	Item 1.2 (rede); Item 1.3 (lado servidor); Item 2.2 (a arquitetura reutiliza a Figura 1)

Uma aplicação Web executa em um ambiente composto por clientes e servidores. O cliente é o dispositivo do usuário e o software que ele utiliza para acessar a Web, geralmente um navegador. Os servidores são computadores que armazenam páginas e aplicações e respondem às solicitações dos clientes (MDN, 2026a). Neste trabalho, o ambiente é descrito a partir do caso de uso da autenticação, no qual o usuário informa e-mail e senha e a aplicação decide se o acesso é permitido.

O ambiente é formado por cinco componentes: o navegador, o servidor DNS (*Domain Name System*), o servidor web (Nginx), o servidor de aplicação (Spring Boot com Tomcat) e o Sistema de Gerenciamento de Banco de Dados (*SGBD*) PostgreSQL. A Figura 1 apresenta esses componentes e suas conexões, e a Figura 2 mostra a sequência de mensagens trocadas durante uma tentativa de autenticação.

Figura 1 – Diagrama de implantação do ambiente Web

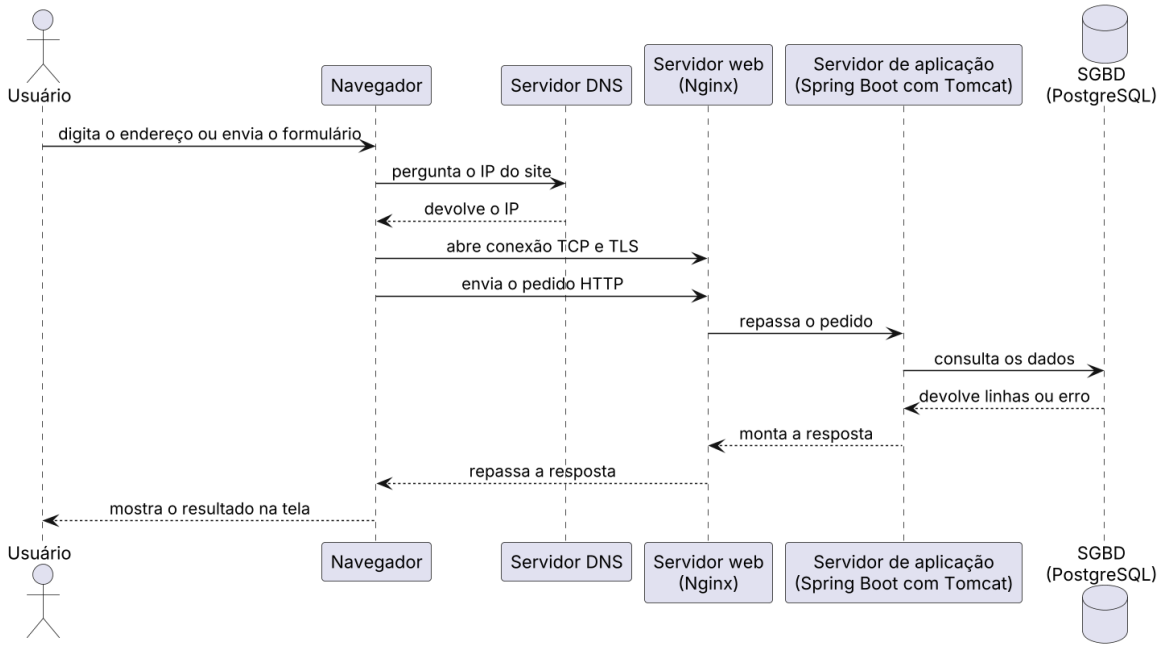


Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026).

Nota: as portas são as padrão e podem ser configuradas (FIELDING et al., 2022; SPRING, s.d.; POSTGRESQL, s.d.).

O fluxo se inicia no navegador. Para localizar o servidor da aplicação, ele consulta o servidor DNS, responsável por traduzir o nome do site em um endereço IP (*Internet Protocol*) (MDN, 2026a). Com o endereço, o navegador envia uma requisição HTTP (*Hypertext Transfer Protocol*), isto é, uma mensagem estruturada que contém o método, o destino e os dados da solicitação, com as credenciais do usuário. O servidor web recebe a requisição e a repassa ao servidor de aplicação, que valida as credenciais consultando o *SGBD*. A resposta percorre o caminho inverso até o navegador. Os protocolos de rede envolvidos são detalhados no Item 1.2, e os componentes do lado servidor, no Item 1.3.

Figura 2 – Diagrama de sequência geral de uma requisição Web



Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026).

No lado cliente, o navegador pode ser acionado de três formas. Na primeira, o usuário digita o endereço da aplicação, e o navegador envia uma requisição *GET* para obter a página (MDN, 2026a). Na segunda, o usuário preenche o formulário de login e o envia. Com o método *POST*, os dados seguem no corpo da requisição e não na URL (*Uniform Resource Locator*), o que evita expor a senha na barra de endereço (MDN, 2026c). Na terceira, um código JavaScript da própria página envia a requisição por meio da Fetch API (*Application Programming Interface*). Essa interface, disponível no navegador, permite que o código da página faça requisições HTTP e processe as respostas sem recarregar a página, e substitui o antigo

XMLHttpRequest (MDN, 2026e). Como o fetch não trata uma resposta de erro HTTP, como 401, como falha da chamada, o código deve verificar explicitamente o status da resposta (MDN, 2026e).

Ao receber a resposta, o navegador interpreta o HTML (*HyperText Markup Language*) e o CSS (*Cascading Style Sheets*) e constrói o DOM (*Document Object Model*) e o CSSOM (*CSS Object Model*), que são as representações em árvore do conteúdo e dos estilos da página. Em seguida, combina ambos em uma árvore de renderização, calcula o tamanho e a posição de cada elemento (layout) e desenha o resultado na tela (MDN, 2026b). No caso da autenticação, o resultado exibido ao usuário é a página de acesso concedido ou a mensagem de erro. Adicionalmente, o navegador pode armazenar cookies enviados pelo servidor no cabeçalho Set-Cookie e reenviá-los nas requisições seguintes ao mesmo servidor (MDN, 2026d). Neste software, o servidor não envia cookies, porque não mantém sessão (ver Item 2.1). A Tabela 1 resume as três formas de invocação do navegador, o que ele processa em cada uma e o que produz.

Tabela 1 – Navegador: invocação, processamento e resultado

Forma de invocação	Quem o invoca e como	O que ele processa	O que ele produz
Endereço digitado	O usuário, ao digitar o endereço da aplicação	A resposta HTTP com HTML, CSS e JavaScript da página	Requisição GET enviada; DOM e CSSOM; página de login renderizada
Formulário de login	O usuário, ao preencher e enviar o formulário	A resposta HTTP à requisição POST	Requisição POST com as credenciais no corpo; nova página de sucesso ou de erro renderizada; cookies armazenados (se houver; neste software em questão não há)
JavaScript (fetch)	Código JavaScript da própria página, via Fetch API	O objeto de resposta devolvido pela chamada, incluindo o status	Requisição HTTP enviada sem recarregar a página; resultado atualizado na tela pelo próprio código

Em síntese, o navegador é o ponto de partida do fluxo: ele coleta as credenciais informadas pelo usuário, monta e envia a requisição HTTP e, ao receber a resposta,

apresenta o resultado na tela. Neste projeto, o navegador não decide se o acesso é permitido. Essa decisão é tomada pelo servidor de aplicação, após consultar o SGBD. A requisição enviada pelo navegador chega primeiro ao servidor web, cuja atuação é descrita no Item 1.3, e o caminho que ela percorre na rede, incluindo DNS, *Transmission Control Protocol* (TCP) e *Transport Layer Security* (TLS), é detalhado no Item 1.2.

Item 1.2 – Comunicação em rede: DNS, TCP, TLS e HTTP

Autoria	Isadora Yasmim da Silva
Revisão	Ana Luísa Pereira dos Santos
Referências usadas	BISHOP (2022); BRAY (2017); EDDY (2022); FIELDING et al. (2022a, 2022b); MDN (2026b, 2026e); MOCKAPETRIS (1987a, 1987b); RESCORLA (2026); Anthropic (2026b), uso de IA registrado no Apêndice VI.
Itens relacionados	Item 1.1 (Figuras 1 e 2); Item 1.3 (lado servidor); Item 2.3 (contrato da API)

O Item 1.1 mostrou que o navegador envia uma requisição HTTP ao servidor web e recebe uma resposta (ver Figura 2). Este item detalha o caminho que essa requisição percorre na rede. Quatro protocolos atuam em sequência: o DNS localiza o servidor, o TCP abre o canal de transporte, o TLS protege esse canal e o HTTP transporta a requisição e a resposta. Para cada um, descreve-se quem o invoca e como, o que ele processa e o que produz. A Figura 3 apresenta a sequência de mensagens em uma tentativa de autenticação, e a Tabela 2 resume os quatro protocolos.

Resolução de nomes (DNS). O DNS é um sistema distribuído que associa nomes de domínio a informações como endereços IP (MOCKAPETRIS, 1987a). Ele é invocado quando o navegador precisa do endereço do servidor a partir do nome contido na URL. O navegador, diretamente ou por meio do sistema operacional, aciona um resolvedor, que envia a consulta a um servidor DNS, normalmente pela porta 53 (MOCKAPETRIS, 1987b). Se a resposta estiver em cache, ela é devolvida imediatamente. Caso contrário, o servidor consulta outros servidores de nomes até obter a resposta de quem tem autoridade sobre o domínio. Cada registro traz um

tempo de vida, o TTL (*Time To Live*), que limita por quanto tempo ele pode permanecer em cache (MOCKAPETRIS, 1987a). O resultado é o endereço IP do servidor web (Nginx). Se o nome não existir, o resolvedor devolve um erro, e a requisição não chega a ser enviada.

Conexão de transporte (TCP). Com o endereço IP, o sistema operacional da máquina do usuário abre uma conexão TCP com o Nginx na porta 443, porta padrão do esquema https (FIELDING et al., 2022a), chamado de HTTPS (HTTP sobre TLS). O navegador não executa o TCP diretamente: ele solicita a conexão ao sistema operacional, que implementa o protocolo. O TCP oferece às aplicações um serviço de fluxo de bytes confiável e ordenado (EDDY, 2022). A conexão é aberta pelo *three-way handshake*: o cliente envia um segmento SYN (*synchronize*, sincronizar) com seu número de sequência inicial; o servidor responde com SYN e ACK (*acknowledgment*, confirmação), confirmando esse número e informando o seu; e o cliente confirma com ACK (EDDY, 2022). Durante a transmissão, o TCP numera os bytes, retransmite segmentos perdidos e entrega os dados na ordem em que foram enviados. O resultado é um canal confiável e ordenado entre o navegador e o Nginx, sobre o qual atuam o TLS e o HTTP.

Segurança do canal (TLS). O TCP não protege o conteúdo transmitido, que pode ser lido por qualquer equipamento no caminho. Por isso, logo após abrir a conexão, o navegador inicia o TLS na versão 1.3, protocolo que fornece autenticação do servidor, confidencialidade e integridade dos dados (RESCORLA, 2026). No handshake completo, que exige uma única ida e volta, o navegador envia a mensagem ClientHello com as versões e os algoritmos que suporta e com sua parte da troca de chaves. O Nginx responde com ServerHello, que contém sua parte da troca de chaves. As mensagens seguintes do servidor já trafegam cifradas: as extensões (EncryptedExtensions), o certificado do servidor (Certificate), uma assinatura que comprova a posse da chave privada correspondente (CertificateVerify) e a mensagem Finished (RESCORLA, 2026). O certificado vincula a chave pública do servidor ao nome do domínio (MDN, 2026e). O navegador verifica se o certificado é válido para o domínio acessado (FIELDING et al., 2022a) e confere a assinatura e a mensagem Finished recebidas (RESCORLA, 2026). Em seguida, os dois lados derivam as chaves de sessão, e o navegador envia o seu

Finished. O resultado é um canal cifrado e autenticado: terceiros não conseguem ler os dados nem alterá-los sem que a alteração seja detectada.

No caso da autenticação, o TLS é essencial porque a senha trafega em texto no corpo da requisição POST (ver Item 1.1). Sem o TLS, qualquer equipamento no trecho externo, entre o navegador e o servidor, poderia lê-la. O uso de HTTP sobre TLS corresponde ao esquema https (FIELDING et al., 2022a). Neste ambiente, o TLS termina no Nginx: é ele que possui o certificado, conclui o handshake e decifra a requisição. O trecho interno entre o Nginx e o servidor de aplicação usa HTTP na porta 8080, no mesmo servidor, em endereço local, e é descrito no Item 1.3.

Transferência das mensagens (HTTP). Com o canal seguro estabelecido, o navegador envia a requisição HTTP, acionada pelo envio do formulário ou pelo código JavaScript da página (ver Item 1.1). O HTTP é um protocolo de requisição e resposta: o cliente envia uma requisição e o servidor devolve uma resposta (FIELDING et al., 2022a). No HTTP/1.1, cada mensagem é formada por uma linha inicial, pelos cabeçalhos, por uma linha em branco e por um corpo opcional (FIELDING et al., 2022b; MDN, 2026b). Na requisição, a linha inicial contém o método, o alvo e a versão do protocolo. Na resposta, ela contém a versão, o código de status e uma frase explicativa. Os cabeçalhos são pares nome e valor que descrevem a mensagem, como Host, que indica o domínio, Content-Type, que indica o formato do corpo, e Content-Length, que indica o tamanho do corpo em bytes. Este trabalho considera o HTTP/1.1 sobre TCP e TLS. O HTTP/2 muda a forma de transmissão sobre o mesmo TCP, e o HTTP/3 usa outro protocolo de transporte, o QUIC, no lugar do TCP. Ambas as versões preservam os métodos, os cabeçalhos e os códigos de status (MDN, 2026b; BISHOP, 2022) e ficam fora do escopo deste trabalho.

O método indica a ação solicitada. O GET pede uma representação do recurso indicado e é usado para obter a página de login. O POST pede que o servidor processe o conteúdo enviado no corpo, e é usado para enviar as credenciais (FIELDING et al., 2022a). O código de status, por sua vez, informa o resultado em três dígitos, cujo primeiro indica a classe: 2xx para sucesso, 4xx para erro do cliente e 5xx para erro do servidor. Na API de autenticação são previstos quatro códigos principais, conforme o contrato do Item 2.3: 200 (OK), quando a requisição foi atendida; 400 (Bad Request), quando a requisição está malformada; 401

(Unauthorized), quando faltam credenciais de autenticação válidas; e 500 (Internal Server Error), quando uma falha inesperada no servidor impede o atendimento (FIELDING et al., 2022a). O Item 2.3 lista também os códigos que o Spring gera quando o método, o tipo de conteúdo ou a rota não são aceitos (405, 415 e 404). O corpo das mensagens de autenticação usa o JSON (*JavaScript Object Notation*), formato textual que representa objetos como pares nome e valor entre chaves, identificado pelo tipo de mídia application/json no cabeçalho Content-Type (BRAY, 2017).

Exemplo comentado. Na autenticação, o navegador envia a requisição abaixo ao Nginx, já dentro do canal TLS. O domínio e os dados são fictícios.

```
POST /api/autenticacao HTTP/1.1
Host: aplicacao.example
Content-Type: application/json
Content-Length: 59

{"email": "maria@aplicacao.example", "senha": "S3nh@Forte"}
```

A primeira linha informa o método POST, o alvo /api/autenticacao e a versão HTTP/1.1. O cabeçalho Host identifica o domínio, e Content-Type e Content-Length indicam que o corpo é um JSON de 59 bytes. Após a linha em branco, o corpo traz o e-mail e a senha, legíveis apenas nas pontas do canal, o navegador e o Nginx, que decifra o canal TLS. Se as credenciais forem válidas, a resposta é:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 69

{"id": 42, "email": "maria@aplicacao.example", "nome": "Maria Souza"}
```

A linha de status indica sucesso, e o corpo contém a identificação do usuário, sem a senha. Se o e-mail não existir ou a senha estiver incorreta, a resposta é a mesma nos dois casos:

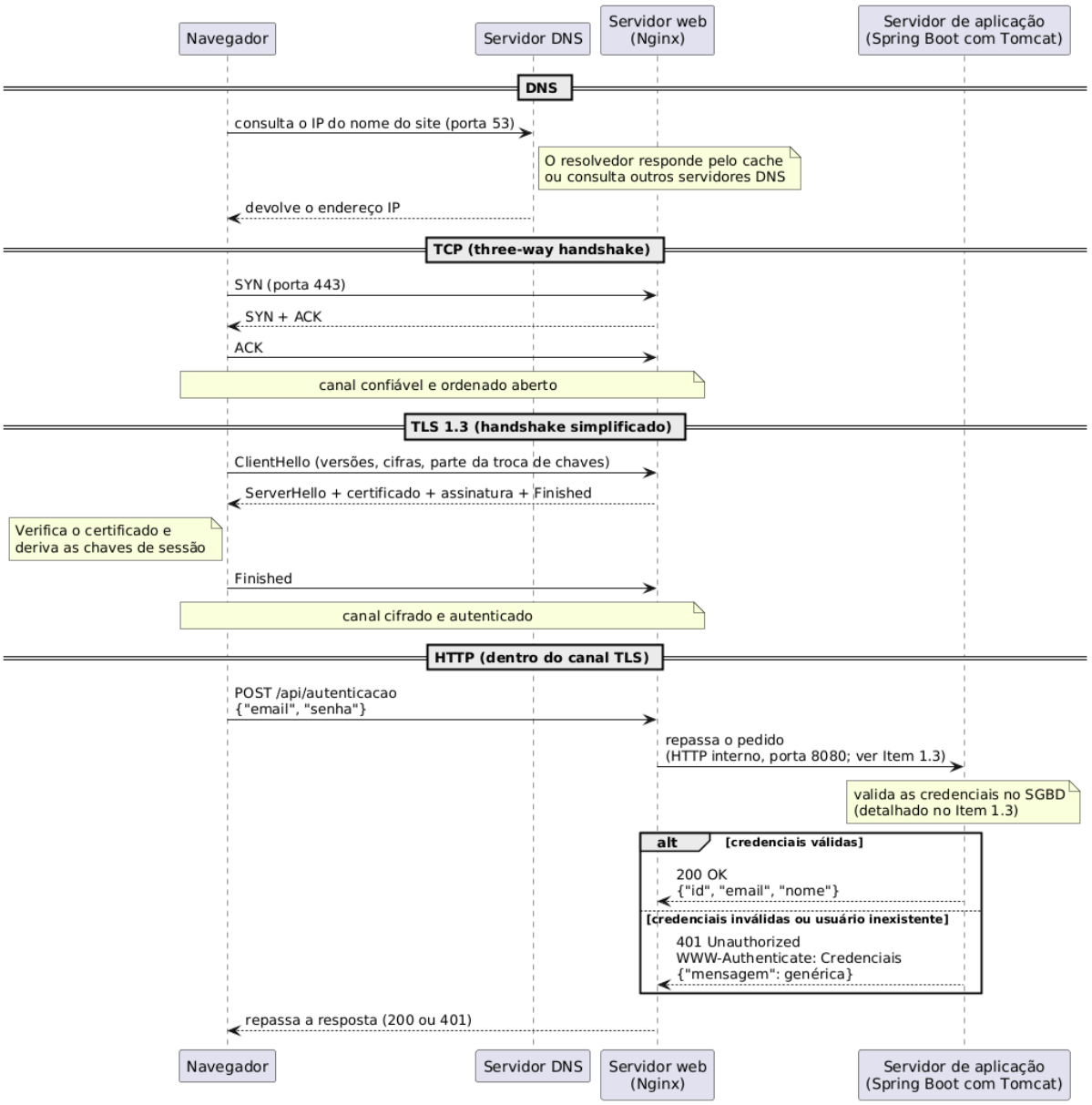
```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Credenciais realm="aplicacao.example"
Content-Type: application/json
Content-Length: 43

{"mensagem": "E-mail ou senha inválidos."}
```

A mensagem é genérica e não informa qual dos dois dados está errado. O cabeçalho WWW-Authenticate indica o esquema de autenticação da aplicação e é

exigido em toda resposta 401 (FIELDING et al., 2022a). A escolha desse esquema é explicada no Item 2.3.

Figura 3 – Diagrama de sequência da comunicação em rede na autenticação



Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026b).

A Figura 3 reúne as etapas descritas: a consulta DNS, o *three-way handshake* TCP, o handshake TLS simplificado e a requisição POST, que o Nginx repassa ao servidor de aplicação. As respostas 200 e 401 saem do servidor de aplicação e voltam ao navegador pelo Nginx. A Tabela 2 resume, para cada protocolo, quem o invoca e como, o que ele processa e o que produz.

Tabela 2 – Protocolos de rede: invocação, processamento e resultado

Protocolo	Quem o invoca e como	O que ele processa	O que ele produz
DNS	O navegador, diretamente ou via sistema operacional, aciona o resolvedor, que consulta um servidor DNS (porta 53)	O nome do domínio contido na URL; respostas em cache ou obtidas de outros servidores de nomes	O endereço IP do servidor web (Nginx), ou erro se o nome não existir
TCP	O sistema operacional, a pedido do navegador, por meio do three-way handshake (SYN, SYN + ACK, ACK) com o Nginx na porta 443	Os segmentos trocados, com números de sequência, confirmações e retransmissões	Um canal confiável e ordenado entre o navegador e o Nginx
TLS 1.3	O navegador, logo após a abertura da conexão TCP, com a mensagem ClientHello	A troca de chaves, o certificado do servidor e sua assinatura, e as mensagens Finished	Um canal cifrado e autenticado, encerrado no Nginx, com chaves de sessão
HTTP	O navegador, ao enviar o formulário ou pelo código JavaScript da página (ver Item 1.1), com a requisição POST /api/autenticacao	A linha inicial, os cabeçalhos e o corpo JSON com e-mail e senha	Uma resposta com código de status (200, 400, 401 ou 500) e corpo JSON

Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026b).

Em síntese, a requisição de autenticação depende de quatro protocolos encadeados: o DNS fornece o endereço, o TCP abre o canal, o TLS o protege e o HTTP transporta a requisição e a resposta. O alcance da rede pública termina no Nginx, onde o TLS é encerrado e a requisição chega decifrada. A partir desse ponto, o Nginx repassa a requisição ao servidor de aplicação (Spring Boot com Tomcat), que consulta o SGBD (PostgreSQL) por JDBC (*Java Database Connectivity*) na porta 5432. Esse processamento no lado servidor é detalhado no Item 1.3.

Item 1.3 – Lado servidor: servidor web, servidor de aplicação e banco de dados

Autoria	Davi Augusto Ferreira de Carvalho
Revisão	Isadora Yasmim da Silva
Referências usadas	APACHE SOFTWARE FOUNDATION (s.d.); FIELDING et al. (2022a); NGINX (s.d.a, s.d.b, s.d.c); OWASP (s.d.); POSTGRESQL (s.d.a, s.d.b, s.d.c, s.d.d); SPRING (s.d.a, s.d.b, s.d.c, s.d.d, s.d.e, s.d.f); Anthropic (2026c), uso de IA registrado no Apêndice VI.

Autoria	Davi Augusto Ferreira de Carvalho
Itens relacionados	Item 1.1 (Figuras 1 e 2); Item 1.2 (TLS encerrado no Nginx e códigos de status); Item 2.2 (implantação dos componentes); Item 2.3 (tabela usuario e verificação da senha)

O Item 1.2 mostrou que a requisição de autenticação chega ao Nginx pela porta 443, onde o TLS é encerrado e a requisição é decifrada. A partir desse ponto, três softwares atuam em sequência: o servidor web (Nginx), o servidor de aplicação (Spring Boot com Tomcat) e o SGBD (PostgreSQL). Para cada um, descreve-se como é iniciado, quem o invoca, o que processa e o que produz. A Figura 4 mostra o atendimento de um recurso estático, a Figura 5 mostra o processamento da autenticação, e a Tabela 3 resume os três componentes.

Servidor web (Nginx)

O Nginx é iniciado pelo sistema operacional como um serviço. Um processo mestre lê e avalia o arquivo de configuração e mantém os processos de trabalho (*worker processes*), que são os que de fato processam as requisições (NGINX, s.d.a). Neste ambiente, o Nginx escuta duas portas. Na porta 80, ele apenas redireciona para o endereço https, de modo que a página de login seja sempre carregada pelo canal cifrado e o formulário envie a senha por https. O cabeçalho Strict-Transport-Security (HSTS) complementa o redirecionamento: ele faz o navegador converter as próximas tentativas de acesso por http em https, sem passar pela porta 80 (MDN, 2026h). Na porta 443, o Nginx possui o certificado do domínio e encerra o TLS, aliviando o servidor de aplicação do custo da criptografia (NGINX, s.d.c).

A cada requisição, o Nginx é invocado pelo navegador, por meio da conexão TCP e TLS descrita no Item 1.2. Ele escolhe o bloco `server` correspondente ao domínio e compara o caminho da URL com os blocos `location` configurados, selecionando o de prefixo mais longo (NGINX, s.d.a). O resultado depende do tipo de recurso solicitado:

- **Recurso estático.** Para o GET da página de login (ver Item 1.1), o Nginx junta o caminho da URL ao diretório indicado pela diretiva `root` e devolve o arquivo HTML, CSS ou JavaScript diretamente do disco. Se o arquivo não existir, responde 404 (NGINX, s.d.a).
- **Recurso dinâmico.** Para o POST /api/autenticacao, o Nginx atua como *proxy* reverso: a diretiva `proxy_pass` repassa a requisição ao servidor de aplicação,

recebe a resposta e a devolve ao navegador (NGINX, s.d.a). Por padrão, ele substitui o cabeçalho Host pelo endereço do servidor de destino; por isso, a configuração o redefine com o domínio original (NGINX, s.d.b).

Como o Nginx abre uma nova conexão com o servidor de aplicação, este vê como origem o endereço local, e não o do navegador. E, como o TLS termina no Nginx, vê também o esquema http. Por isso, o Nginx acrescenta os cabeçalhos X-Forwarded-For e X-Forwarded-Proto, com o IP do cliente e o esquema original, e os sobrescreve em vez de repassar valores vindos de fora, que poderiam ser forjados. O Spring Boot só usa esses cabeçalhos quando configurado com a propriedade `server.forward-headers-strategy` (SPRING, s.d.f), mostrada mais adiante.

Exemplo comentado. O trecho abaixo mostra a configuração do Nginx para este ambiente. O domínio e os caminhos são fictícios.

```
server {
    listen 80;
    server_name aplicacao.example;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name aplicacao.example;
    ssl_certificate     /etc/nginx/tls/aplicacao.crt;
    ssl_certificate_key /etc/nginx/tls/aplicacao.key;
    ssl_protocols       TLSv1.3;
    add_header Strict-Transport-Security "max-age=31536000" always;

    location / {
        root /var/www/aplicacao;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host              $host;
        proxy_set_header X-Forwarded-For   $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
```

```
    }
}
```

O primeiro bloco redireciona a porta 80 para https. O segundo aponta o certificado usado no handshake TLS (ver Item 1.2), envia o cabeçalho HSTS, serve a página de login pelo `location /` e repassa a API pelo `location /api/`, mantendo o caminho original /api/autenticacao (NGINX, s.d.b).

Além dos códigos da API definidos no Item 1.2 (200, 400, 401 e 500), o Nginx gera códigos próprios da infraestrutura: 301 no redirecionamento, 404 para um arquivo estático inexistente e 502 (*Bad Gateway*) ou 504 (*Gateway Timeout*) quando o servidor de aplicação está parado ou não responde a tempo. Os dois últimos são definidos para servidores que atuam como *proxy* (FIELDING et al., 2022a).

Servidor de aplicação (Spring Boot com Tomcat)

A aplicação é iniciada como um programa Java, e o Spring Boot sobe um Tomcat embutido, que escuta por padrão a porta 8080 (SPRING, s.d.a). No Tomcat, quem recebe as conexões é o conector HTTP, componente que escuta uma porta TCP e encaminha as requisições ao processamento interno. Por padrão, o conector escuta todos os endereços da máquina (APACHE SOFTWARE FOUNDATION, s.d.). Como seu único cliente é o Nginx, instalado na mesma máquina (ver Figura 1), a aplicação é configurada para escutar apenas o endereço local, e o trecho interno pode usar HTTP sem TLS:

```
server.address=127.0.0.1
server.port=8080
server.forward-headers-strategy=native
```

A primeira linha restringe o conector ao endereço local, e a terceira faz o Tomcat usar os cabeçalhos X-Forwarded-For e X-Forwarded-Proto enviados pelo Nginx (SPRING, s.d.f).

A cada requisição, o servidor de aplicação é invocado pelo Nginx, que abre uma conexão HTTP com a porta 8080 e envia a requisição repassada. O processamento segue quatro etapas:

1. **Conector e thread.** O conector aceita a conexão e atribui à requisição uma thread de um conjunto de threads, que a acompanha até o fim do processamento (APACHE SOFTWARE FOUNDATION, s.d.).

2. **Servlet.** Servlet é um componente web Java, gerenciado por um contêiner, que gera conteúdo dinâmico; o contêiner, como o Tomcat, decodifica as requisições e formata as respostas (ECLIPSE FOUNDATION, 2024). O Tomcat converte a mensagem HTTP nos objetos `HttpServletRequest` e `HttpServletResponse` e os entrega ao DispatcherServlet, o controlador frontal (*front controller*) do Spring, que delega o trabalho a componentes configuráveis (SPRING, s.d.b).
3. **Mapeamento e conversão.** O HandlerMapping associa POST /api/autenticacao ao método do código da aplicação, e o HandlerAdapter o invoca (SPRING, s.d.c). Antes de invocar o método, um HttpMessageConverter converte o corpo JSON em objeto Java (SPRING, s.d.d). JSON malformado resulta em 400 (SPRING, s.d.g); campos ausentes ou vazios são rejeitados com 400 pela validação (`@Valid` com `@NotBlank` e `@Email`), definida no Item 2.3 (SPRING, s.d.d).
4. **Execução.** O código da aplicação busca o usuário pelo e-mail no SGBD, compara a senha com o hash armazenado e decide o resultado (ver Item 2.3).

O resultado é uma resposta HTTP com código de status e corpo JSON: o objeto devolvido pelo método é convertido em JSON, e o Tomcat escreve a resposta na conexão aberta pelo Nginx. Exceções não tratadas pelo código, como uma falha no SGBD, são capturadas por um tratador de exceções da aplicação (classe anotada com `@RestControllerAdvice`), acionado pelo HandlerExceptionResolver (SPRING, s.d.g). Esse tratador monta a resposta 500 no formato do contrato (ver Item 2.3). Sem ele, o Spring Boot devolveria seu próprio corpo de erro, pela rota /error, fora do contrato (SPRING, s.d.h).

SGBD (PostgreSQL)

O PostgreSQL também é iniciado como um serviço do sistema operacional. Um processo supervisor, o *postmaster*, escuta a porta TCP configurada, por padrão a 5432 (POSTGRESQL, s.d.a). O servidor segue o modelo de um processo por conexão: a cada pedido de conexão, o postmaster cria um processo de servidor dedicado (*backend*), que atende aquele cliente até a conexão ser encerrada (POSTGRESQL, s.d.c).

O SGBD é invocado pelo servidor de aplicação por meio do driver JDBC (*Java Database Connectivity*), que implementa o protocolo de comunicação do PostgreSQL (POSTGRESQL, s.d.c). As conexões vêm de um *pool*, por padrão o

HikariCP, que as mantém abertas e as reutiliza entre requisições, evitando criar um processo de backend a cada login (SPRING, s.d.e).

Com a conexão emprestada, a aplicação envia a consulta que busca o usuário pelo e-mail:

```
SELECT id, email, nome, hash_senha FROM usuario WHERE email = ?
```

O símbolo ? é um parâmetro. Com uma consulta parametrizada (PreparedStatement), o e-mail digitado é enviado separado do texto *Structured Query Language*, linguagem de consulta estruturada (SQL) e tratado como dado, e não como parte do comando. Isso impede que um valor malicioso no campo de e-mail altere a consulta, ataque conhecido como injeção de SQL (OWASP, s.d.). A tabela usuario é definida no Item 2.3.

No processo de backend, a consulta passa por quatro etapas: o analisador verifica a sintaxe e gera uma árvore de consulta; o sistema de reescrita aplica as regras armazenadas no catálogo; o planejador estima o custo dos caminhos possíveis, como usar o índice da coluna email, e escolhe o mais barato; e o executor percorre o plano, lê as tabelas e devolve as linhas resultantes (POSTGRESQL, s.d.b).

O resultado produzido pelo SGBD é um de três: uma linha com os dados do usuário, quando o e-mail existe; nenhuma linha, quando não existe; ou um erro, identificado por um código de cinco caracteres no padrão SQLSTATE, cujos dois primeiros indicam a classe do erro (POSTGRESQL, s.d.d). No driver JDBC, as linhas chegam como um ResultSet, e o erro, como uma exceção (SQLException). Em seguida, a conexão volta ao pool para ser reutilizada pela próxima requisição.

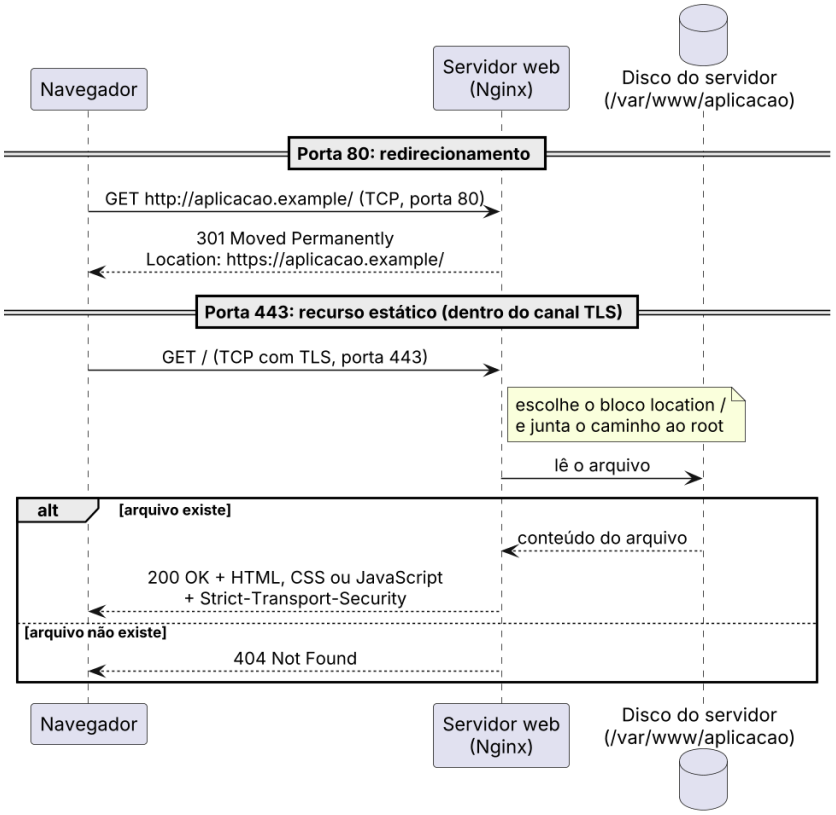
O acesso ao SGBD também é restrito. Como mostra a Figura 1, o PostgreSQL fica fora do servidor da aplicação, em outra máquina da rede interna. O parâmetro `listen_addresses` define em quais endereços ele aceita conexões; o padrão, localhost, só aceita a própria máquina, então ele é configurado com o endereço da rede interna (POSTGRESQL, s.d.a). O arquivo `pg_hba.conf` define quais máquinas, bancos e usuários podem se conectar e com qual método de autenticação; conexões que não correspondem a nenhuma regra são recusadas (POSTGRESQL, s.d.e). Neste ambiente, a regra aceita apenas o servidor da aplicação, com o usuário

de banco da aplicação, que tem só os privilégios necessários, como a leitura da tabela usuario.

Caminho de volta

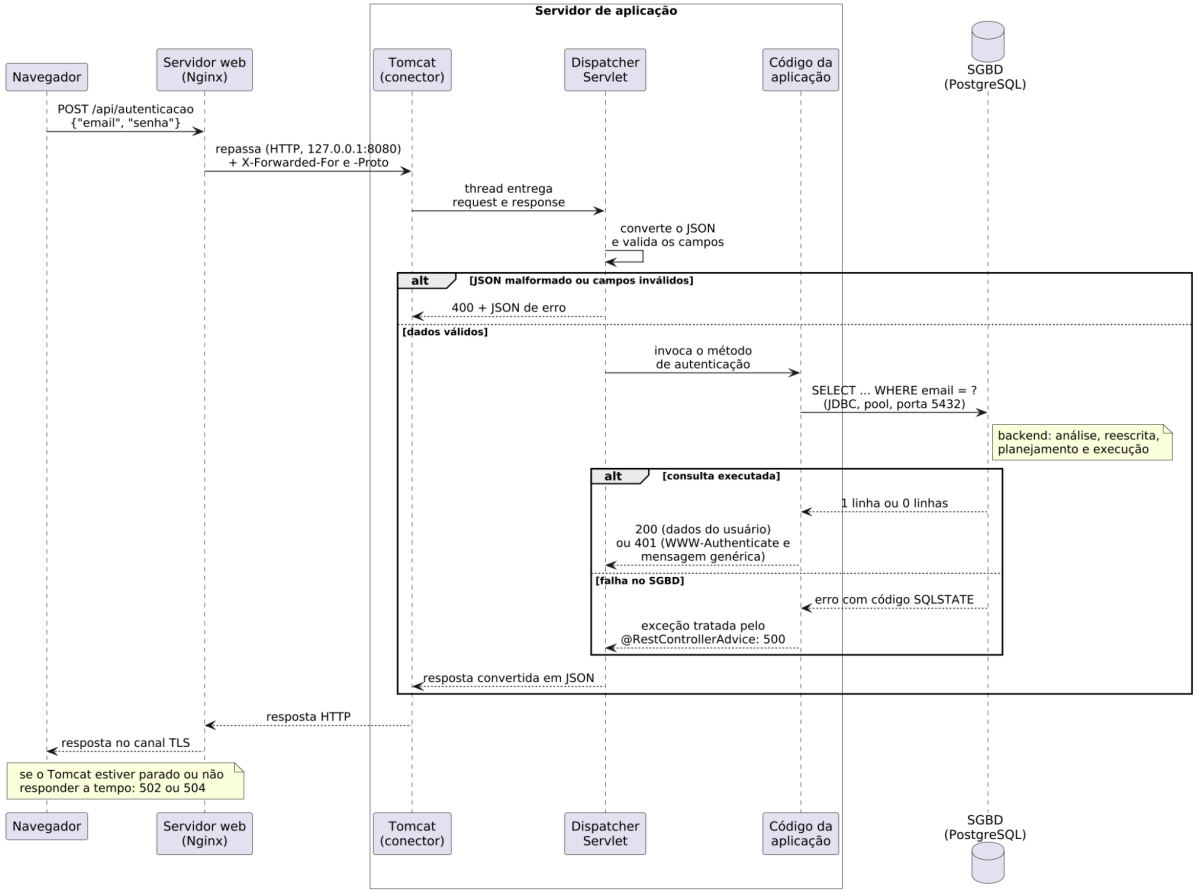
A resposta percorre o caminho inverso: do SGBD à aplicação, que decide o resultado, e dela ao Tomcat e ao Nginx, que a envia ao navegador pelo canal TLS (ver Item 1.2). A Figura 4 mostra o redirecionamento da porta 80 e o atendimento de um recurso estático, que não passa pelo servidor de aplicação. A Figura 5 mostra o processamento completo da autenticação, incluindo os caminhos do 400 e do 500.

Figura 4 – Diagrama de sequência do atendimento de um recurso estático pelo Nginx



Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026c).

Figura 5 – Diagrama de sequência do processamento da autenticação no lado servidor



Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026c).

A Tabela 3 resume, para cada software do lado servidor, quem o invoca e como, o que ele processa e o que produz.

Tabela 3 – Lado servidor: invocação, processamento e resultado

Componente	**Quem o invoca e como**	**O que ele processa**	**O que ele produz**
Servidor web (Nginx)	Iniciado pelo sistema operacional como serviço; a cada requisição, pelo navegador, via TCP na porta 80 (só redirecionamento) e TCP com TLS na porta 443	A requisição decifrada, comparada com os blocos location: caminhos estáticos são lidos do disco e /api/ é repassado com proxy_pass	Arquivo estático (200 ou 404); requisição repassada ao servidor de aplicação com os cabeçalhos X-Forwarded; resposta da aplicação devolvida ao navegador; 301 na porta 80; 502 ou 504 se a aplicação falhar
Servidor de aplicação (Spring Boot com Tomcat)	Iniciado como programa Java, que sobe o Tomcat em 127.0.0.1, porta 8080; a cada requisição, pelo Nginx, via HTTP interno	A requisição HTTP, entregue por uma thread ao DispatcherServlet, que converte e valida o corpo JSON e invoca o método da aplicação	Consulta enviada ao SGBD; resposta HTTP com status 200, 400, 401 ou 500 e corpo JSON

Componente	Quem o invoca e como	O que ele processa	O que ele produz
SGBD (PostgreSQL)	Iniciado como serviço, com o postmaster escutando a porta 5432 na rede interna; a cada consulta, pelo servidor de aplicação, via driver JDBC e pool de conexões	A consulta SQL parametrizada, que passa por análise, reescrita, planejamento e execução no processo de backend	Uma linha com os dados do usuário, nenhuma linha ou um erro com código SQLSTATE

Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026c).

Em síntese, o Nginx protege a borda, serve os arquivos estáticos e repassa a API; o Spring Boot com Tomcat executa o código da aplicação; e o PostgreSQL guarda e devolve os dados do usuário. Cada um só aceita conexões de quem está imediatamente antes dele no fluxo: o Tomcat escuta apenas o endereço local, e o PostgreSQL aceita apenas o servidor da aplicação. A organização do código da aplicação em camadas é tratada no Item 2.2.

Seção 2 – Projeto da aplicação Web de autenticação

Item 2.1 – Escopo e requisitos

Autoria	Isadora Yasmim da Silva

Revisão	Ana Luísa Pereira dos Santos
Referências usadas	ISO/IEC/IEEE (2018, 2026); OWASP (s.d.b, s.d.c); SOMMERVILLE (2018); TEMOSHOK et al. (2025); Anthropic (2026d), uso de IA registrado no Apêndice VI.
Itens relacionados	Item 1.2 (HTTPS e códigos de status); Item 1.3 (validação do corpo, tratador de exceções e consulta parametrizada); Item 2.2 (arquitetura que realiza o RF01); Item 2.3 (contrato da API, tabela usuario e verificação da senha)

A Seção 1 descreveu o ambiente em que a aplicação executa: o navegador, a rede e os softwares do lado servidor. Este item inicia a Seção 2 e define o que o software faz e o que fica fora dele. No ciclo de vida, a definição dos requisitos antecede e orienta o projeto da arquitetura e o desenho detalhado (ISO/IEC/IEEE, 2026). Os requisitos funcionais descrevem os serviços que o software oferece. Os não funcionais impõem restrições sobre esses serviços (SOMMERVILLE, 2018). Cada requisito foi redigido para ser necessário, singular e verificável (ISO/IEC/IEEE, 2018).

Requisito funcional

O software tem um único requisito funcional, que corresponde ao domínio definido no Apêndice A do enunciado.

RF01 – Autenticar usuário. O software deve receber e-mail e senha, verificar se correspondem a um usuário registrado e devolver a identificação desse usuário em caso de sucesso, ou um erro de autenticação em caso de falha. O RF01 se decompõe nos quatro passos do Apêndice A:

1. **Entrada das credenciais (RF01.1).** O servidor de aplicação recebe o e-mail e a senha no corpo da requisição POST /api/autenticacao (ver Item 1.2). Antes de prosseguir, verifica se a requisição está bem formada (JSON válido; campos e-mail e senha presentes e não vazios; e-mail no formato de endereço, verificado pela anotação @Email). Se não estiver, responde 400 e não executa os passos seguintes, pois não há credencial a avaliar (ver Item 1.3).
2. **Validação da existência do usuário (RF01.2).** O servidor de aplicação busca no SGBD o usuário com o e-mail informado, na tabela usuario (ver Item 2.3).

3. **Comparação da credencial (RF01.3).** Se o usuário existe, o servidor de aplicação verifica a senha informada contra o hash registrado na coluna hash_senha. A senha nunca é comparada em texto, pois não é armazenada (ver RNF02 na Tabela 5).
4. **Decisão (RF01.4).** Se o usuário existe e a senha confere, o resultado é *autenticado*, e a resposta é 200 com um objeto que contém id, e-mail e nome. Se o usuário não existe ou a senha não confere, o resultado é *não autenticado*, e a resposta é 401, com o mesmo cabeçalho WWW-Authenticate e a mesma mensagem genérica nos dois casos. O formato das respostas é definido no Item 2.3.

Caso de uso

Um caso de uso descreve a interação entre um ator externo e o sistema para atingir um objetivo (SOMMERVILLE, 2018). A Figura 6 mostra o caso de uso "Autenticar usuário", que realiza o RF01, e a Tabela 4 descreve seus fluxos. O ator é o usuário, que interage com o sistema pelo navegador. O servidor web (Nginx), o servidor de aplicação (Spring Boot com Tomcat) e o SGBD (PostgreSQL) fazem parte do sistema e, por isso, não aparecem como atores.

Figura 6 – Diagrama de caso de uso da aplicação de autenticação



Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026d).

Tabela 4 – Caso de uso "Autenticar usuário"

Campo	Descrição
Caso de uso	Autenticar usuário (realiza o RF01)
Ator	Usuário, por meio do navegador
Pré-condições	(1) A página de login foi carregada pelo canal HTTPS (ver Item 1.3). (2) Os usuários válidos já estão registrados na tabela usuario por meio externo a este software, pois o cadastro está fora do escopo.

Campo	Descrição
Pós-condição de sucesso	O cliente recebe id, e-mail e nome do usuário e pode continuar o fluxo. Nenhum dado é alterado no SGBD, e nenhuma sessão é criada no servidor.
Pós-condição de falha	O cliente recebe um código de erro. A resposta não indica qual credencial está errada nem se o e-mail existe, e nenhum dado é alterado.
Fluxo principal	1. O usuário informa e-mail e senha no formulário e confirma o envio. 2. O navegador envia POST /api/autenticacao pelo canal HTTPS (ver Item 1.2). 3. O servidor de aplicação verifica se o corpo está bem formado (RF01.1). 4. O servidor de aplicação busca o usuário pelo e-mail no SGBD e encontra uma linha (RF01.2). 5. O servidor de aplicação verifica a senha contra o hash registrado, e ela confere (RF01.3). 6. O servidor de aplicação decide *autenticado* e responde 200 com a identificação do usuário (RF01.4; ver Item 2.3). 7. O navegador apresenta o resultado ao usuário.
FA1 – Dados malformados	No passo 3, o JSON está malformado ou um campo está ausente, vazio ou com e-mail inválido. O servidor de aplicação responde 400 (ver Item 2.3) sem consultar o SGBD, e o caso de uso termina.
FA2 – Usuário inexistente	No passo 4, a consulta não devolve nenhuma linha. O servidor de aplicação decide *não autenticado* e responde 401 com o cabeçalho WWW-Authenticate e a mensagem genérica (ver Item 2.3), em tempo semelhante ao do FA3 (RNF06). O caso de uso termina.
FA3 – Senha incorreta	No passo 5, a senha não confere com o hash. O servidor de aplicação decide *não autenticado* e responde 401 com o mesmo cabeçalho e a mesma mensagem do FA2 (ver Item 2.3). O caso de uso termina.
FE1 – Falha no SGBD	No passo 4, o SGBD está indisponível ou devolve erro. O tratador de exceções responde 500 sem expor detalhes internos (ver Item 1.3 e Item 2.3). O caso de uso termina sem decisão de autenticação.

Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026d).

Requisitos não funcionais

Foram registrados apenas os requisitos não funcionais que afetam o projeto, apresentados na Tabela 5. Os detalhes de implementação ficam nos itens indicados na última coluna.

Tabela 5 – Requisitos não funcionais

Identificador	Requisito	Justificativa	Onde é tratado
RNF01	As credenciais trafegam apenas por HTTPS entre o navegador e o servidor web.	A senha vai em texto no corpo da requisição. A página de login e o envio das credenciais devem usar TLS (OWASP, s.d.b), e a senha deve ser enviada por canal protegido e autenticado (TEMOSHOK et al., 2025).	Item 1.2 (TLS); Item 1.3 (redirecionamento da porta 80 e HSTS)
RNF02	A senha é armazenada como hash com sal, gerado por algoritmo adaptativo, o Argon2id.	Algoritmos lentos e com sal dificultam a força bruta e as tabelas pré-computadas mesmo que a tabela vaze (OWASP, s.d.c). A senha não deve ser armazenada em texto (TEMOSHOK et al., 2025).	Item 2.3 (algoritmo, parâmetros e coluna hash_senha)
RNF03	A resposta para usuário inexistente e para senha incorreta é idêntica: status 401, o mesmo cabeçalho WWW-Authenticate e a mesma mensagem.	Respostas diferentes permitem descobrir quais e-mails estão registrados, ataque conhecido como enumeração de contas (OWASP, s.d.b).	Tabela 4 (FA2 e FA3); Item 2.3 (contrato)
RNF04	Nenhuma resposta contém a senha ou o hash.	Um hash exposto permite ataque de força bruta fora do sistema, sem limite de tentativas (OWASP, s.d.c). O cliente precisa apenas da identificação do usuário.	Item 2.3 (objeto de resposta)
RNF05	A consulta ao banco é parametrizada.	Impede que o valor do campo de e-mail altere o comando SQL (ver Item 1.3).	Item 1.3 (consulta parametrizada); Item 2.3 (repositório)
RNF06	O tempo de resposta para usuário inexistente é semelhante ao de senha incorreta.	A verificação do hash é lenta de propósito (RNF02). Se ela for omitida quando o usuário não existe, a resposta fica mais rápida e revela a existência do e-mail, o que anula o RNF03 (OWASP, s.d.b).	Item 2.3 (verificação contra um hash fictício quando o usuário não existe)

Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026d).

Fora do escopo

O Apêndice A limita o software à verificação de credenciais. Em caso de sucesso, o objeto devolvido contém apenas id, e-mail e nome, o suficiente para o cliente continuar o fluxo com o usuário autenticado. Ficam fora do escopo:

- **Cadastro de usuário:** criar contas é outra funcionalidade; o software apenas lê usuários já registrados.
- **Recuperação ou troca de senha:** altera a credencial registrada, enquanto o software apenas a compara.
- **Logout:** só existe quando há sessão, e o software não cria sessão.
- **Criação e gestão de sessão ou emissão de token:** o Apêndice A pede apenas a identificação do usuário; o que o cliente faz depois com ela não faz parte deste software.
- **Autorização e perfis:** decidir o que o usuário pode fazer é posterior a saber quem ele é.
- **Autenticação multifator:** exigiria um segundo fator além de e-mail e senha, que são as credenciais definidas para este projeto.
- **Bloqueio e limitação de tentativas:** exigem registrar o estado das tentativas, e o software não altera dados no SGBD.
- **Login social:** a credencial seria verificada por um provedor externo, e não comparada com a credencial registrada neste software.

Em síntese, o software tem um requisito funcional, o RF01, realizado pelo caso de uso "Autenticar usuário" com um fluxo de sucesso, três fluxos alternativos e um de exceção (ver Tabela 4), sob as restrições da Tabela 5. O Item 2.2 organiza os componentes que realizam esse caso de uso nos nós do ambiente da Seção 1. O Item 2.3 detalha o contrato da API, a tabela usuario e a verificação da senha.

Item 2.2 – Arquitetura da aplicação

Autoria	Ana Luísa Pereira dos Santos
Revisão	Davi Augusto Ferreira de Carvalho
Referências usadas	MICROSOFT (2023); SOMMERVILLE (2018); SPRING (s.d.g, s.d.i, s.d.j); ANTHROPIC (2026e), uso de IA registrado no Apêndice VI.
Itens relacionados	Item 1.1 (Figura 1); Item 1.3 (Figura 5); Item 2.1 (RF01, Tabela 4 e Tabela 5); Item 2.3 (classes, contrato da API e tabela usuario)

O Item 2.1 definiu o que o software faz: receber e-mail e senha e devolver a identificação do usuário ou um erro de autenticação. Este item define como o software é organizado para fazer isso. A arquitetura de um software descreve como o sistema é organizado e qual é a sua estrutura geral (SOMMERVILLE, 2018, cap. 6). Aqui, ela responde a duas perguntas: quais são as partes do software e onde cada uma executa no ambiente descrito na Seção 1.

Estilo arquitetural

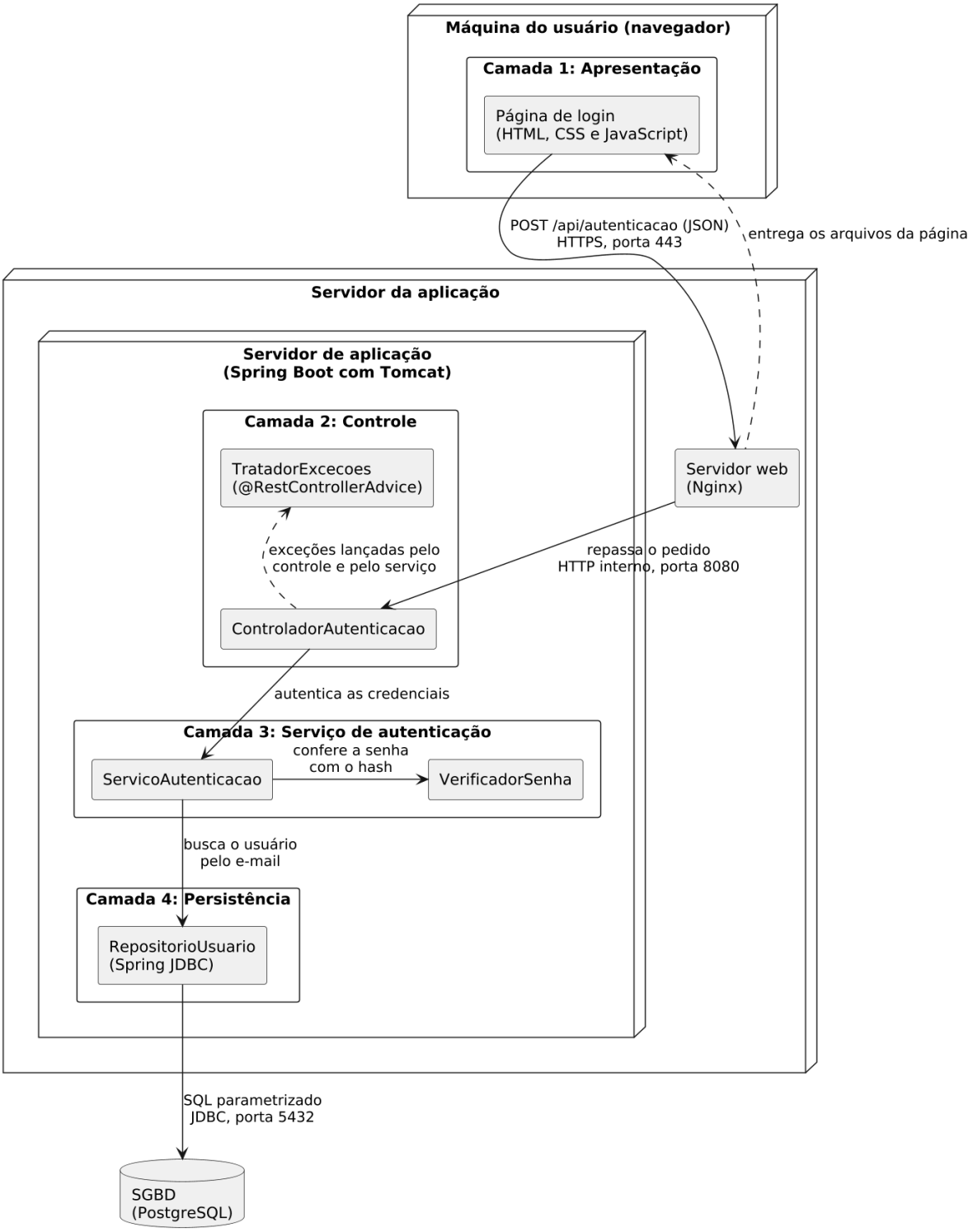
O software segue o estilo cliente-servidor: o navegador, no lado do cliente, envia requisições, e o servidor as processa e responde (ver Item 1.1). No lado servidor, o código da aplicação é organizado em camadas. Nesse padrão, cada camada oferece serviços à camada acima e usa os serviços da camada abaixo (SOMMERVILLE, 2018, cap. 6). A vantagem é que cada camada tem uma responsabilidade só, e uma mudança em uma delas afeta pouco as outras.

Todas as camadas do servidor fazem parte de um único programa Java construído com Spring Boot, implantado como uma unidade em um só servidor. Um software assim é chamado de monólito: o comportamento principal executa em um único processo, e a aplicação inteira é implantada como uma unidade (MICROSOFT, 2023). Juntando as duas ideias, o estilo deste projeto é o monólito em camadas: monólito porque o servidor é um programa único, e em camadas porque o código interno é dividido por responsabilidade. A escolha é proporcional ao escopo: o software tem um único requisito funcional (RF01) e não há outros serviços que justifiquem dividi-lo em programas separados.

Camadas e componentes

A Figura 7 combina a visão de componentes com a de implantação: cada componente aparece dentro do nó da Figura 1 em que executa (ver Item 1.1). Os nós são os mesmos da Figura 1, exceto o DNS, que não executa componentes do software: a máquina do usuário, o servidor da aplicação (com o Nginx e o Spring Boot com Tomcat) e o SGBD. A fronteira entre cliente e servidor passa entre a camada de apresentação, que executa no navegador, e as outras três camadas, que executam no servidor de aplicação. Camadas são uma separação lógica do código, e não uma separação física em máquinas (MICROSOFT, 2023). Por isso a apresentação é uma camada mesmo executando no navegador.

Figura 7 – Diagrama de componentes e implantação da aplicação de autenticação



Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026e).

Cada camada tem uma responsabilidade. A apresentação mostra o formulário de login, envia e-mail e senha e exibe o resultado (ver Item 1.1). O controle recebe a requisição, converte o JSON em objeto, valida o formato e devolve a resposta (ver Item 1.3). O serviço de autenticação aplica a regra do RF01: busca o usuário, confere a senha e decide o

resultado. A conferência da senha é feita pelo VerificadorSenha, um componente auxiliar do serviço, e não uma quinta camada. A persistência é o único ponto de acesso ao banco: o RepositorioUsuario busca o usuário pelo e-mail. A Tabela 6 resume cada camada e os requisitos do Item 2.1 que ela atende.

Tabela 6 – Camadas: responsabilidade, componentes, local de execução e requisitos

Camada	Responsabilidade	Componentes	Onde executa (Figura 1)	Requisitos (Item 2.1)
Apresentação	Mostrar o formulário, enviar as credenciais e exibir o resultado	Página de login (HTML, CSS e JavaScript)	Navegador, na máquina do usuário; arquivos servidos pelo Nginx	RF01.1
Controle	Receber a requisição, converter o JSON, validar o formato e devolver a resposta	ControladorAutenticacao e TratadorExcecoes	Servidor de aplicação (Spring Boot com Tomcat)	RF01.1; RF01.4; RNF03; RNF04
Serviço de autenticação	Buscar o usuário, conferir a senha e decidir o resultado	ServicoAutenticacao e VerificadorSenha	Servidor de aplicação (Spring Boot com Tomcat)	RF01.2 a RF01.4; RNF02; RNF06
Persistência	Buscar o usuário no SGBD pelo e-mail	RepositorioUsuario	Servidor de aplicação; os dados ficam no SGBD (PostgreSQL)	RF01.2; RNF05

Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026e).

Adotamos como regra que cada camada chame apenas a camada imediatamente abaixo. A arquitetura em camadas permite impor essa restrição (MICROSOFT, 2023), e a regra mantém as responsabilidades separadas e o desenho simples. As interfaces entre as camadas são as setas da Figura 7: o pedido HTTP com JSON entre o navegador e o controle, a chamada de autenticação do controle ao serviço, a busca do usuário por e-mail do serviço ao repositório e o SQL parametrizado do repositório ao PostgreSQL. Os nomes e as operações de cada componente são detalhados no Item 2.3.

Um exemplo com o login mostra como as camadas trabalham juntas. Quando o usuário envia o formulário, o pedido chega ao Nginx, que o repassa ao servidor de aplicação; o Tomcat e o DispatcherServlet o entregam ao ControladorAutenticacao (ver Item 1.3, Figura 5). O controle valida o formato e chama o ServicoAutenticacao. O serviço pede ao RepositorioUsuario o usuário com aquele e-mail e usa o VerificadorSenha para comparar a senha recebida com o hash armazenado. Se a comparação der certo, o serviço devolve a identificação do usuário, e o controle a envia ao navegador. Caso contrário, o serviço sinaliza a falha de autenticação. O caso do usuário inexistente, que deve levar um tempo

semelhante ao da senha incorreta (RNF06), é detalhado no Item 2.3. Esses passos correspondem ao RF01.1 a RF01.4.

Tratamento de erros

O serviço não conhece o protocolo HTTP, então não monta respostas. Ele só informa o resultado: devolve o usuário ou sinaliza a falha de autenticação. Quem monta a resposta é um tratador global de exceções, na camada de controle, implementado com uma classe anotada com @RestControllerAdvice, que vale para todos os controladores (SPRING, s.d.i). Ele converte cada tipo de erro conhecido em uma resposta no formato do contrato (ver Item 2.3): o formato inválido vira 400, a falha de autenticação recebe sempre a mesma resposta 401, seja o usuário inexistente ou a senha incorreta (RNF03), e a falha inesperada, como a queda do SGBD, vira 500. O tratador também recebe as exceções que o Spring gera ao converter e validar o JSON, antes de o pedido chegar ao controlador (SPRING, s.d.g), e é isso que produz o 400. Assim, o código de status de cada situação fica definido em um único lugar.

Decisões arquiteturais

A Tabela 7 reúne as decisões que moldam a arquitetura e a justificativa de cada uma.

Tabela 7 – Decisões arquiteturais e justificativas

Decisão	Justificativa	Fundamentação
Monólito em camadas: cliente-servidor, com o servidor em camadas e em um único programa	Cada camada tem uma responsabilidade só. O escopo tem um único requisito funcional, sem outros serviços que peçam programas separados.	MICROSOFT (2023); SOMMERVILLE (2018, cap. 6); RF01
Verificar a senha apenas no servidor	O hash fica no servidor e não é enviado ao navegador. Verificar no cliente exigiria expô-lo.	RNF02; RNF04
O Nginx encerra o TLS e repassa o pedido ao Spring Boot por HTTP, em endereço local	A proteção da rede fica concentrada na borda, e o trecho interno não sai do servidor.	RNF01; Itens 1.2 e 1.3
API em JSON, com um único endpoint: POST /api/autenticacao	A página estática só precisa enviar um POST e ler a resposta, o que mantém o contrato simples.	Itens 1.2 e 2.3
Tratamento de erros centralizado em um tratador global	O código de status de cada situação fica em um só lugar, e as respostas para usuário inexistente e senha incorreta saem idênticas.	SPRING (s.d.g, s.d.i); RNF03
Acesso ao banco por Spring JDBC (`JdbcTemplate`), com consulta parametrizada	A consulta parametrizada impede que o e-mail altere o comando SQL, e o SQL fica visível no código, como na Figura 5.	SPRING (s.d.j); RNF05
Persistência isolada no RepositorioUsuario	Só uma camada conhece o SQL e a tabela usuario, e o serviço não depende do banco.	Tabela 6

Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026e).

Em síntese, o software é um monólito em camadas. A apresentação é servida pelo Nginx e executa no navegador. O controle, o serviço de autenticação e a persistência formam um único programa Java, construído com Spring Boot, que executa no servidor de aplicação, atrás do Nginx, com os dados no PostgreSQL. O software não mantém sessão (ver Item 2.1), então cada requisição de autenticação é decidida só com os dados que ela traz. O Item 2.3 detalha as classes de cada camada, o contrato da API e a verificação da senha.

Item 2.3 – Desenho detalhado dos componentes

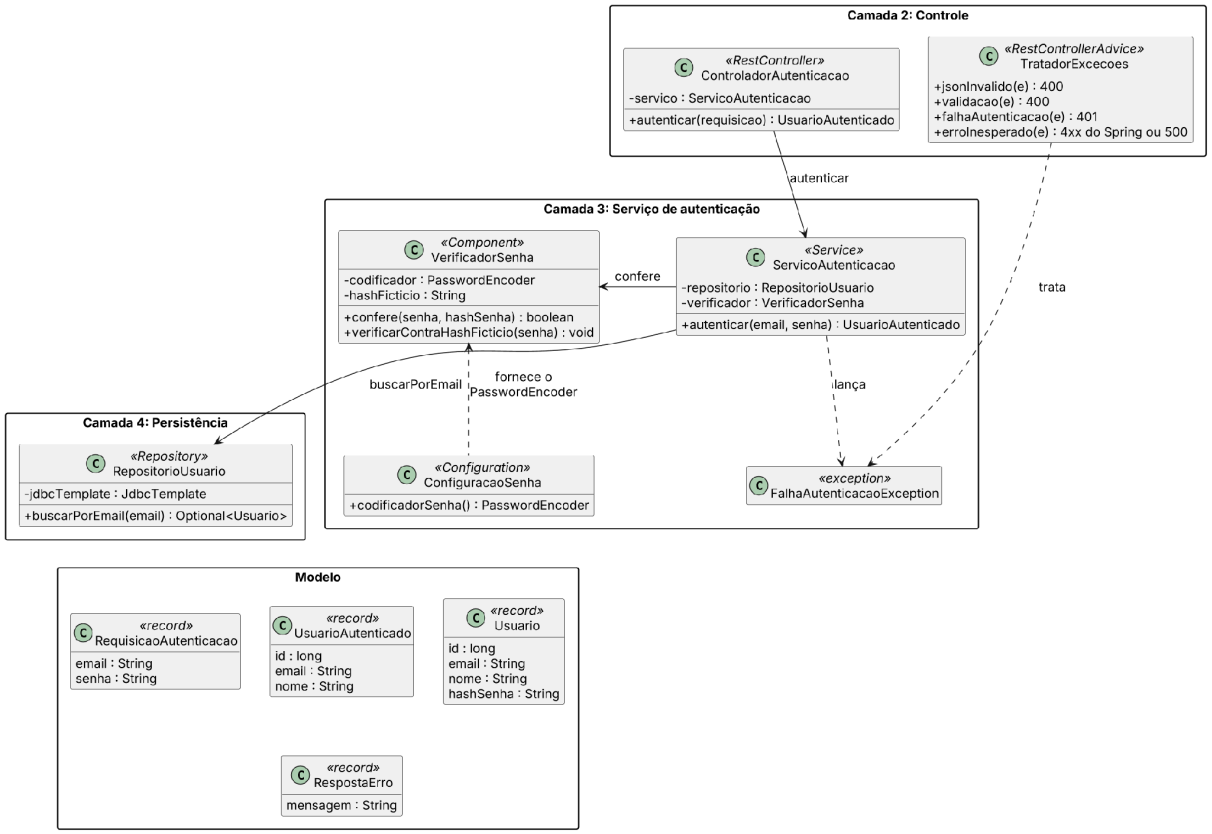
Campo	**Conteúdo**
Autoria	Davi Augusto Ferreira de Carvalho
Revisão	Isadora Yasmim da Silva
Referências usadas	BRAY (2017); FIELDING et al. (2022a); OWASP (s.d.b, s.d.c); POSTGRESQL (s.d.f); SOMMERVILLE (2018); SPRING (s.d.d, s.d.g, s.d.i, s.d.j, s.d.k); TEMOSHOK et al. (2025); ANTHROPIC (2026f), uso de IA registrado no Apêndice VI, Seção VI.7.
Itens relacionados	Item 1.2 (mensagens HTTP e códigos de status); Item 1.3 (Figura 5, tratador de exceções e consulta parametrizada); Item 2.1 (RF01, Tabela 4 e Tabela 5); Item 2.2 (Figura 7 e Tabela 6)

O Item 2.2 dividiu o software em quatro camadas e nomeou seus componentes. Este item detalha cada componente do servidor: as classes e suas operações, o contrato da API, a sequência de chamadas em cada fluxo do caso de uso, a tabela usuario e o algoritmo que verifica a senha. A camada de apresentação, que executa no navegador, já foi descrita no Item 1.1. Os diagramas usam a notação *Unified Modeling Language*, linguagem de modelagem unificada (UML): o diagrama de classes mostra as classes e suas associações, e o diagrama de sequência mostra as interações entre os objetos ao longo do tempo (SOMMERVILLE, 2018, cap. 5).

Classes e responsabilidades

A Figura 8 mostra as classes do servidor agrupadas pelas camadas do Item 2.2. As setas cheias são as chamadas entre as classes, e as tracejadas indicam outras dependências: o serviço lança a FalhaAutenticacaoException, o tratador a trata, e a ConfiguracaoSenha fornece o codificador ao VerificadorSenha. Cada camada chama só a camada imediatamente abaixo, como definido no Item 2.2. O pacote Modelo reúne os objetos de dados trocados entre as camadas; eles não têm regra de negócio. Para deixar o diagrama legível, a Tabela 8 indica qual classe cria ou recebe cada um desses objetos, em vez de setas.

Figura 8 – Diagrama de classes da aplicação de autenticação



Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026f).

A Tabela 8 resume a responsabilidade e as operações de cada classe e os requisitos do Item 2.1 que ela atende. As linhas seguem a ordem das camadas: controle, serviço, persistência e modelo. Dois objetos separam o que fica dentro e o que sai do servidor. O Usuario representa a linha da tabela e contém o hash da senha, por isso nunca é enviado ao navegador. O UsuarioAutenticado tem só id, e-mail e nome e é o único objeto devolvido em caso de sucesso (RNF04).

Tabela 8 – Classes: responsabilidade, operações e requisitos

Classe	**Responsabilidade**	**Operações principais**	**Requisitos**
ControladorAutenticacao	Recebe POST /api/autenticacao, exige JSON válido e campos no formato (`@Valid`) e repassa as credenciais ao serviço	`autenticar(requisicao)` : devolve UsuarioAutenticado	RF01.1; RF01.4
TratadorExcecoes	Classe anotada com `@RestControllerAdvice`, a variante de `@ControllerAdvice` que escreve o retorno no corpo da resposta (SPRING, s.d.i). É o único lugar que	JSON malformado e validação → 400; falha de autenticação → 401 com WWW-Authenticate; método, Content-Type ou rota não aceitos → mantém o status do Spring (405, 415 ou 404); qualquer outra → 500	RF01.4; RNF03; RNF04

Classe	Responsabilidade	Operações principais	Requisitos
	define os códigos de erro e monta o corpo de erro		
ServicoAutenticacao	Aplica a regra do RF01: busca o usuário, manda conferir a senha e decide. Não conhece HTTP	`autenticar(email, senha)`: devolve UsuarioAutenticado ou lança FalhaAutenticacaoException	RF01.2 a RF01.4; RNF03; RNF06
VerificadorSenha	Auxiliar do serviço. Compara a senha recebida com o hash armazenado, sem comparar texto	`confere(senha, hashSenha)`: devolve verdadeiro ou falso; `verificarContraHashFicticio(senha)`: mesmo custo, resultado descartado	RF01.3; RNF02; RNF06
ConfiguracaoSenha	Configuração do serviço. Define o algoritmo e os parâmetros do hash, num só lugar	`codificadorSenha()`: fornece o codificador Argon2id	RNF02
FalhaAutenticacaoException	Exceção única para usuário inexistente e senha incorreta; o controle não consegue distinguir os dois casos	—	RNF03
RepositorioUsuario	Único ponto de acesso ao SGBD, com uma consulta parametrizada por meio do `JdbcTemplate` (SPRING, s.d.j)	`buscarPorEmail(email)`: devolve um Optional com o Usuario, vazio se não houver linha	RF01.2; RNF05
RequisicaoAutenticacao	Corpo da requisição: email (`@NotBlank`, `@Email`) e senha (`@NotBlank`). Recebido pelo controlador	—	RF01.1
UsuarioAutenticado	Corpo da resposta de sucesso: id, email e nome. Criado pelo serviço	—	RF01.4; RNF04
Usuario	Linha da tabela usuario, com o hash; uso interno. Criado pelo repositório	—	RF01.2; RNF04
RespostaErro	Corpo de todas as respostas de erro: mensagem. Criado pelo tratador	—	RNF03; RNF04

Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026f).

As classes de modelo são *records* do Java, objetos imutáveis que só carregam dados. O RequisicaoAutenticacao e o Usuario escondem a senha e o hash na sua representação em texto, para que esses valores não apareçam em registros de log, o que complementa o RNF04.

Contrato da API

A API tem uma única operação: POST /api/autenticacao, com o cabeçalho Content-Type: application/json e um corpo JSON com os campos email e senha (BRAY, 2017). A resposta também é JSON, e o código de status indica o resultado (FIELDING et al., 2022a). A Tabela 9 lista as respostas possíveis. Todas as respostas de erro têm o mesmo formato, com um único campo mensagem, e nenhuma contém a senha, o hash ou detalhes internos (RNF04).

Tabela 9 – Respostas de POST /api/autenticacao

Situação	Status	Corpo e cabeçalhos da resposta	Quem gera
Usuário existe e a senha confere	200 OK	`{"id": 42, "email": "maria@aplicacao.example", "nome": "Maria Souza"}`	ControladorAutenticacao
JSON malformado, corpo ausente, campo ausente ou vazio, ou e-mail fora do formato (FA1)	400 Bad Request	`{"mensagem": "Requisição inválida."}`	TratadorExcecoes
Usuário inexistente (FA2) ou senha incorreta (FA3)	401 Unauthorized	Cabeçalho `WWW-Authenticate: Credenciais realm="aplicacao.example"` e corpo `{"mensagem": "E-mail ou senha inválidos."}`, idênticos nos dois casos (RNF03)	TratadorExcecoes
SGBD indisponível ou outra falha inesperada (FE1)	500 Internal Server Error	`{"mensagem": "Erro interno. Tente novamente mais tarde."}`	TratadorExcecoes
Método diferente de POST, Content-Type diferente de JSON ou rota inexistente	405, 415 ou 404	`{"mensagem": "Requisição inválida."}`, com os cabeçalhos do Spring (no 405, o Allow)	TratadorExcecoes
Servidor de aplicação parado ou sem resposta	502 ou 504	Página de erro do Nginx	Nginx (ver Item 1.3)

Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026f).

A mensagem do 401 é genérica, sem dizer qual credencial está errada, como recomenda a *Open Worldwide Application Security Project* (OWASP) (s.d.b). A Request for Comments (RFC) 9110 exige que toda resposta 401 traga o cabeçalho WWW-Authenticate, com pelo menos um desafio que indique como se autenticar (FIELDING et al., 2022a). Como as credenciais vão no corpo da requisição, e não num cabeçalho HTTP, o desafio usa um esquema próprio da aplicação, Credenciais, com o realm aplicacao.example. O esquema Basic não foi usado porque, ao receber

um 401 com esse desafio, o navegador pode abrir sua própria janela de usuário e senha por cima da página de login (MDN CONTRIBUTORS, 2026).

O TratadorExcecoes também recebe as exceções que o próprio Spring gera quando o método, o Content-Type ou a rota não são aceitos (SPRING, s.d.g). Nesse caso, o método que trata as demais exceções mantém o status e os cabeçalhos do Spring, como o Allow, que a RFC 9110 exige na resposta 405 (FIELDING et al., 2022a), e só troca o corpo pelo formato do contrato. Assim, nenhuma resposta da API sai no formato padrão de erro do Spring Boot.

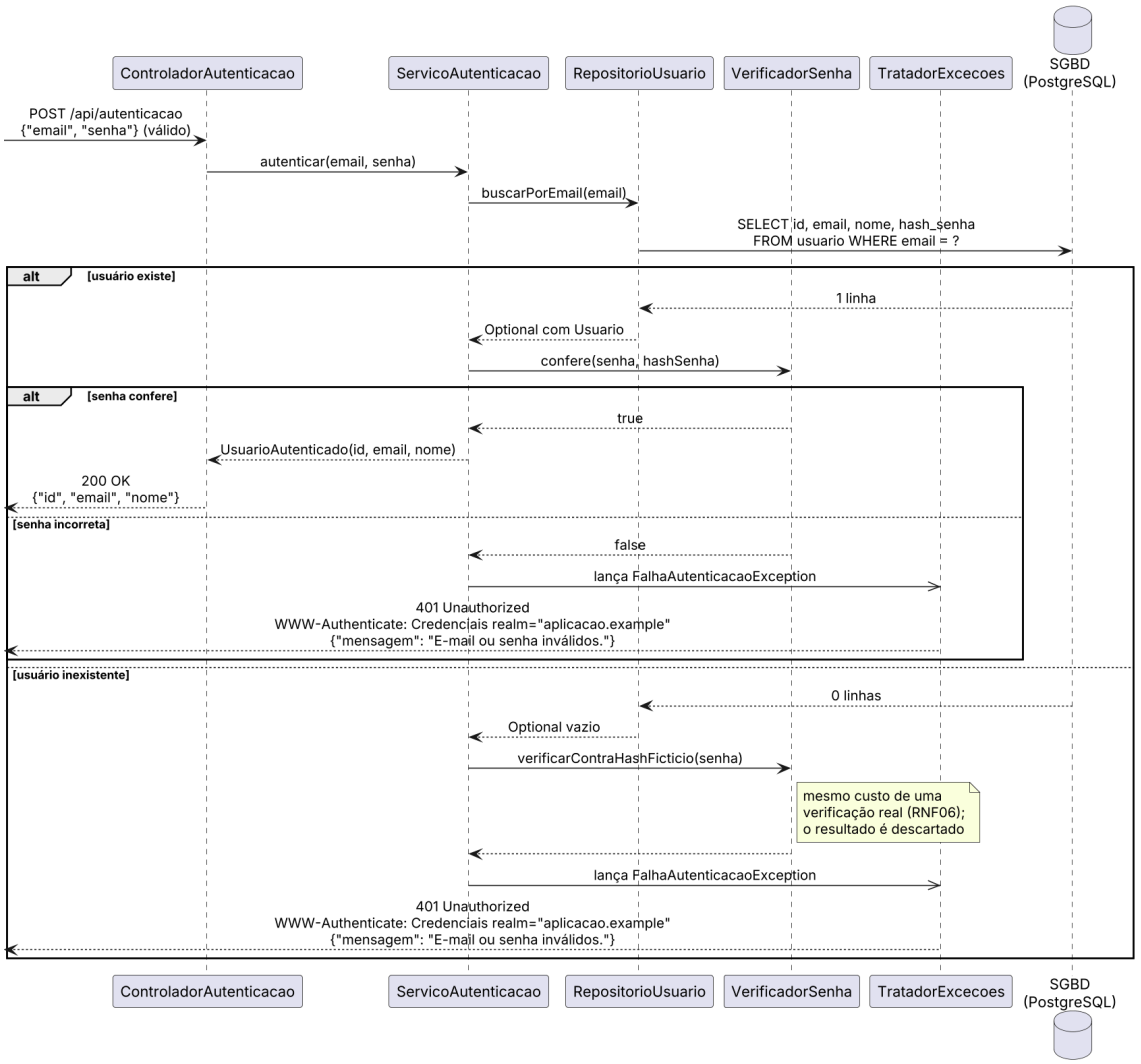
Exemplo comentado. Os exemplos completos da requisição e das respostas 200 e 401, com linha inicial e cabeçalhos, estão no Item 1.2. O caso que o Item 1.2 não mostra é o de dados malformados: com o corpo abaixo, sem a senha e com o e-mail fora do formato, a resposta é 400, e o SGBD não é consultado:
{"email": "maria"}
O corpo da resposta 400 está na Tabela 9.

Diagramas de sequência

As Figuras 9 e 10 mostram as chamadas entre as classes em cada fluxo do caso de uso “Autenticar usuário” (ver Tabela 4, Item 2.1). Elas detalham a parte “Código da aplicação” da Figura 5 (Item 1.3), que mostra o caminho da requisição pelo Nginx, pelo Tomcat e pelo DispatcherServlet. As setas abertas indicam exceções. Na execução, a exceção sobe pela pilha de chamadas até o DispatcherServlet, que a entrega ao TratadorExcecoes (SPRING, s.d.g); os diagramas mostram esse caminho de forma resumida.

A Figura 9 reúne o fluxo principal e os dois fluxos que terminam em 401. A senha incorreta e o usuário inexistente terminam na mesma exceção e na mesma resposta (RNF03). No usuário inexistente, o serviço ainda chama o VerificadorSenha contra um hash fictício, para que a resposta demore o mesmo que a de senha incorreta (RNF06).

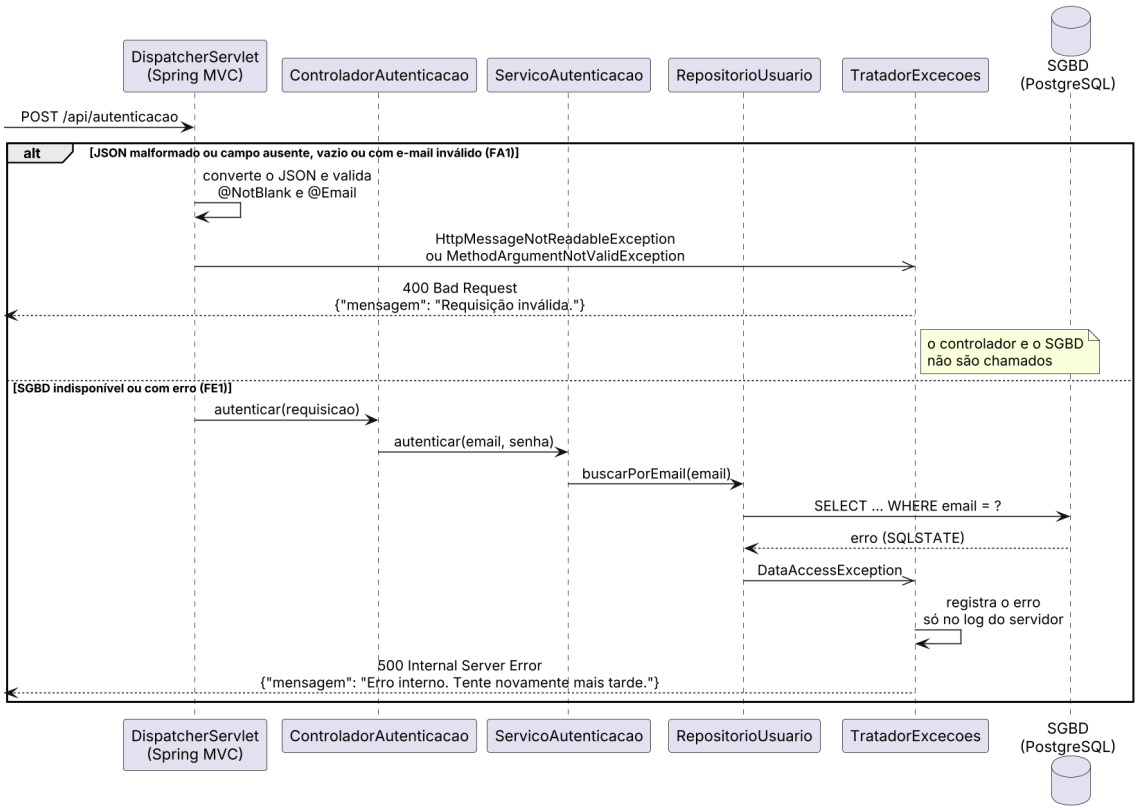
Figura 9 – Diagrama de sequência da autenticação: sucesso, senha incorreta e usuário inexistente



Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026f).

A Figura 10 mostra os fluxos de erro. No FA1, o Spring converte e valida o corpo antes de chamar o controlador; se a conversão ou a validação falham, a exceção vai direto ao TratadorExcecoes, que responde 400 sem que o controlador ou o SGBD sejam chamados (SPRING, s.d.d). No FE1, o erro do SGBD chega ao repositório como uma DataAccessException, e o TratadorExcecoes registra os detalhes apenas no log do servidor e responde 500 com a mensagem genérica.

Figura 10 – Diagrama de sequência dos fluxos de erro: dados malformados e falha no SGBD



Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026f).

Modelo de dados

O software usa uma única tabela, usuario, no banco autenticacao do PostgreSQL. A Tabela 10 descreve suas colunas. A senha não é armazenada: só o hash, na coluna hash_senha (RNF02). Como o cadastro está fora do escopo (ver Item 2.1), as linhas são inseridas por meio externo, por um *script* de carga executado pelo administrador do SGBD.

Tabela 10 – Colunas da tabela usuario

Coluna	Tipo	Restrições	Uso
id	BIGSERIAL	Chave primária	Identifica o usuário na resposta de sucesso
email	VARCHAR(254)	NOT NULL, UNIQUE	Chave de busca da autenticação (RF01.2)
nome	VARCHAR(150)	NOT NULL	Devolvido na resposta de sucesso
hash_senha	VARCHAR(255)	NOT NULL	Hash Argon2id da senha, com o sal e os parâmetros (RF01.3)

Fonte: elaborada pelos autores, com apoio de IA (ANTHROPIC, 2026f).

A restrição UNIQUE garante que um e-mail corresponda a no máximo um usuário, então a consulta do repositório devolve zero ou uma linha. O PostgreSQL cria automaticamente um índice único para essa restrição (POSTGRESQL, s.d.f), e é esse índice que o planejador pode usar na busca por e-mail (ver Item 1.3).

```
CREATE TABLE usuario (
    id          BIGSERIAL PRIMARY KEY,
    email       VARCHAR(254) NOT NULL UNIQUE,
    nome        VARCHAR(150) NOT NULL,
    hash_senha  VARCHAR(255) NOT NULL
);


GRANT CONNECT ON DATABASE autenticacao TO app_autenticacao;
GRANT SELECT ON TABLE usuario TO app_autenticacao;
```

O usuário de banco da aplicação, app_autenticacao, recebe apenas a permissão de conectar ao banco e de ler a tabela usuario. Como o software não altera dados, um erro ou um ataque na aplicação não consegue inserir, alterar ou apagar usuários. Ele ainda pode ler a tabela, inclusive os hashes, e por isso o hash precisa ser forte (RNF02). As regras de rede que limitam o acesso ao SGBD foram descritas no Item 1.3.

Verificação da senha

A senha é armazenada com o Argon2id, algoritmo recomendado pela OWASP para hash de senhas. O projeto usa a configuração mínima indicada por ela: 19 MiB de memória, 2 iterações e paralelismo 1 (OWASP, s.d.c). O Argon2id é lento de propósito e exige muita memória, o que encarece ataques de força bruta mesmo com hardware especializado (SPRING, s.d.k). A implementação é a classe Argon2PasswordEncoder, do Spring Security, criada pela ConfiguracaoSenha.

O *National Institute of Standards and Technology*, instituto de padrões dos Estados Unidos (NIST) exige que a senha seja armazenada com sal e com um esquema adequado de hash de senha, que recebe a senha, o sal e um fator de custo; o sal deve ter pelo menos 32 bits (TEMOSHOK et al., 2025). A ConfiguracaoSenha define um sal aleatório de 16 bytes (128 bits), bem acima desse mínimo. Como o sal é único para cada senha, duas senhas iguais produzem hashes diferentes (OWASP,

s.d.c). O resultado é um texto único que guarda o algoritmo, os parâmetros, o sal e o hash, cada um em sua parte:

```
$argon2id$v=19$m=19456,t=2,p=1$<sal em Base64>$<hash em Base64>
```

A comparação (RF01.3) acontece no método `confere` do VerificadorSenha. O codificador lê o sal e os parâmetros guardados no hash_senha, calcula o hash da senha recebida com esses mesmos valores e compara os dois resultados. A senha nunca é comparada em texto, e o hash armazenado não pode ser revertido para a senha.

O tempo de resposta também precisa ser protegido. Se o serviço respondesse logo quando o e-mail não existe, sem calcular o hash, essa resposta seria bem mais rápida que a de senha incorreta e revelaria quais e-mails estão cadastrados. A OWASP recomenda que a autenticação faça o mesmo processo em todas as tentativas (OWASP, s.d.b). Por isso, o VerificadorSenha gera na inicialização um hash fictício, com o mesmo algoritmo e os mesmos parâmetros dos hashes reais. Quando o usuário não existe, o método `verificarContraHashFicticio` compara a senha recebida com esse hash e descarta o resultado, e o custo fica igual ao de uma verificação real (RNF06).

Os hashes da carga inicial são gerados por um utilitário de linha de comando separado da API, que usa a mesma ConfiguracaoSenha. Assim, os hashes armazenados e os calculados na verificação têm sempre os mesmos parâmetros, e nenhuma rota da API cria ou altera senhas.

Síntese

O servidor tem seis classes com comportamento, uma exceção e quatro objetos de dados, distribuídos nas três camadas do servidor definidas no Item 2.2. O ControladorAutenticacao recebe a requisição, o ServicoAutenticacao decide com a ajuda do VerificadorSenha, o RepositorioUsuario consulta a tabela usuario, e o TratadorExcecoes transforma toda falha em uma resposta do contrato. Os quatro passos do RF01 aparecem nas Figuras 9 e 10: entrada e validação, busca do usuário, comparação do hash e decisão, com 200 no sucesso e 401 idêntico nas duas falhas de credencial.

Referências

ANTHROPIC. Claude. 2026a. Disponível em: https://claude.ai. Acesso em: 24 out. 2026. Prompt: Apêndice VI, Seção VI.2.

ANTHROPIC. Claude. 2026b. Disponível em: https://claude.ai. Acesso em: 25 out. 2026. Prompt: Apêndice VI.

ANTHROPIC. Claude. 2026c. Disponível em: https://claude.ai. Acesso em: 25 out. 2026. Prompt: Apêndice VI.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. (ed.). RFC 9110: HTTP Semantics. IETF, jun. 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9110.html. Acesso em: 24 out. 2026.

MDN CONTRIBUTORS. How the web works. MDN Web Docs, 2026a. Disponível em: https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/How_the_Web_works. Acesso em: 24 out. 2026.

MDN CONTRIBUTORS. Populating the page: how browsers work. MDN Web Docs, 2026b. Disponível em: https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/How_browsers_work. Acesso em: 24 out. 2026.

MDN CONTRIBUTORS. Sending and retrieving form data. MDN Web Docs, 2026c. Disponível em: https://developer.mozilla.org/en-US/docs/Learn/Forms/Sending_and_retrieving_form_data. Acesso em: 24 out. 2026.

MDN CONTRIBUTORS. Using HTTP cookies. MDN Web Docs, 2026d. Disponível em: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies. Acesso em: 24 out. 2026.

MDN CONTRIBUTORS. Using the Fetch API. MDN Web Docs, 2026e. Disponível em: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch. Acesso em: 24 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation: Connections and Authentication. Disponível em: https://www.postgresql.org/docs/current/runtime-config-connection.html. Acesso em: 24 out. 2026.

SPRING. Spring Boot Reference: Embedded Web Servers. Disponível em: https://docs.spring.io/spring-boot/how-to/webserver.html. Acesso em: 24 out. 2026.

BISHOP, M. (ed.). RFC 9114: HTTP/3. IETF, jun. 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9114.html. Acesso em: 26 out. 2026.

BRAY, T. (ed.). RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format. IETF, dez. 2017. Disponível em: https://www.rfc-editor.org/rfc/rfc8259.html. Acesso em: 25 out. 2026.

EDDY, W. (ed.). RFC 9293: Transmission Control Protocol (TCP). IETF, ago. 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9293.html. Acesso em: 25 out. 2026.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. (ed.). RFC 9112: HTTP/1.1. IETF, jun. 2022b. Disponível em: https://www.rfc-editor.org/rfc/rfc9112.html. Acesso em: 25 out. 2026.

MDN CONTRIBUTORS. HTTP messages. MDN Web Docs, 2026b. Disponível em: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages. Acesso em: 25 out. 2026.

MDN CONTRIBUTORS. Transport Layer Security (TLS). MDN Web Docs, 2026e. Disponível em: https://developer.mozilla.org/en-US/docs/Web/Security/Defenses/Transport_Layer_Security. Acesso em: 25 out. 2026.

MOCKAPETRIS, P. RFC 1034: Domain names – concepts and facilities. IETF, nov. 1987a. Disponível em: https://www.rfc-editor.org/rfc/rfc1034.html. Acesso em: 25 out. 2026.

MOCKAPETRIS, P. RFC 1035: Domain names – implementation and specification. IETF, nov. 1987b. Disponível em: https://www.rfc-editor.org/rfc/rfc1035.html. Acesso em: 25 out. 2026.

RESCORLA, E. RFC 9846: The Transport Layer Security (TLS) Protocol Version 1.3. IETF, jul. 2026. Disponível em: https://www.rfc-editor.org/rfc/rfc9846.html. Acesso em: 26 out. 2026.

APACHE SOFTWARE FOUNDATION. Apache Tomcat 10.1 Configuration Reference: The HTTP Connector. Disponível em: https://tomcat.apache.org/tomcat-10.1-doc/config/http.html. Acesso em: 25 out. 2026.

NGINX. Beginner's Guide. nginx.org, s.d.a. Disponível em: https://nginx.org/docs/beginners_guide.html. Acesso em: 25 out. 2026.

NGINX. NGINX Reverse Proxy. NGINX Documentation, s.d.b. Disponível em: https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/. Acesso em: 25 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation: Connections and Authentication. s.d.a. Disponível em: https://www.postgresql.org/docs/current/runtime-config-connection.html. Acesso em: 24 out. 2026.

SPRING. Spring Boot Reference: Embedded Web Servers. s.d.a. Disponível em: https://docs.spring.io/spring-boot/how-to/webserver.html. Acesso em: 24 out. 2026.

SPRING. Spring Boot Reference: SQL Databases. s.d.e. Disponível em: https://docs.spring.io/spring-boot/reference/data/sql.html. Acesso em: 25 out. 2026.

ANTHROPIC. Claude. 2026c. Disponível em: https://claude.ai. Acesso em: 26 out. 2026. Prompt: Apêndice VI, Seção VI.4.

APACHE SOFTWARE FOUNDATION. Apache Tomcat 11 Configuration Reference: The HTTP Connector. s.d. Disponível em: https://tomcat.apache.org/tomcat-11.0-doc/config/http.html. Acesso em: 26 out. 2026.

ECLIPSE FOUNDATION. Jakarta Servlet Specification: version 6.1. 2024. Disponível em: https://jakarta.ee/specifications/servlet/6.1/jakarta-servlet-spec-6.1. Acesso em: 26 out. 2026.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. (ed.). RFC 9110: HTTP Semantics. IETF, jun. 2022a. Disponível em: https://www.rfc-editor.org/rfc/rfc9110.html. Acesso em: 24 out. 2026.

MDN CONTRIBUTORS. Strict-Transport-Security. MDN Web Docs, 2026h. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security. Acesso em: 26 out. 2026.

NGINX. Beginner's Guide. nginx.org, s.d.a. Disponível em: https://nginx.org/docs/beginners_guide.html. Acesso em: 25 out. 2026.

NGINX. NGINX Reverse Proxy. NGINX Documentation, s.d.b. Disponível em: https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/. Acesso em: 25 out. 2026.

NGINX. NGINX SSL Termination. NGINX Documentation, s.d.c. Disponível em: https://docs.nginx.com/nginx/admin-guide/security-controls/terminating-ssl-http/. Acesso em: 25 out. 2026.

OWASP. SQL Injection Prevention Cheat Sheet. OWASP Cheat Sheet Series, s.d. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html. Acesso em: 25 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation: Connections and Authentication. s.d.a. Disponível em: https://www.postgresql.org/docs/current/runtime-config-connection.html. Acesso em: 24 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation: The Path of a Query. s.d.b. Disponível em: https://www.postgresql.org/docs/current/query-path.html. Acesso em: 25 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation: How Connections Are Established. s.d.c. Disponível em: https://www.postgresql.org/docs/current/connect-estab.html. Acesso em: 25 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation: PostgreSQL Error Codes. s.d.d. Disponível em: https://www.postgresql.org/docs/current/errcodes-appendix.html. Acesso em: 25 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation: The pg_hba.conf File. s.d.e. Disponível em: https://www.postgresql.org/docs/current/auth-pg-hba-conf.html. Acesso em: 26 out. 2026.

SPRING. Spring Boot Reference: Embedded Web Servers. s.d.a. Disponível em: https://docs.spring.io/spring-boot/how-to/webserver.html. Acesso em: 24 out. 2026.

SPRING. Spring Framework Reference: DispatcherServlet. s.d.b. Disponível em: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-servlet.html. Acesso em: 25 out. 2026.

SPRING. Spring Framework Reference: Special Bean Types. s.d.c. Disponível em: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-servlet/special-bean-types.html. Acesso em: 25 out. 2026.

SPRING. Spring Framework Reference: @RequestBody. s.d.d. Disponível em: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-methods/requestbody.html. Acesso em: 25 out. 2026.

SPRING. Spring Boot Reference: SQL Databases. s.d.e. Disponível em: https://docs.spring.io/spring-boot/reference/data/sql.html. Acesso em: 25 out. 2026.

SPRING. Spring Security Reference: HTTP (Proxy Server Configuration). s.d.f. Disponível em: https://docs.spring.io/spring-security/reference/features/exploits/http.html. Acesso em: 25 out. 2026.

SPRING. Spring Framework Reference: Exceptions. s.d.g. Disponível em: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-servlet/exceptionhandlers.html. Acesso em: 26 out. 2026.

SPRING. Spring Boot Reference: Servlet Web Applications. s.d.h. Disponível em: https://docs.spring.io/spring-boot/reference/web/servlet.html. Acesso em: 26 out. 2026.

ANTHROPIC. Claude. 2026d. Disponível em: https://claude.ai. Acesso em: 26 out. 2026. Prompt: Apêndice VI, Seção VI.5 [CONFERIR número da seção].

ISO/IEC/IEEE. **ISO/IEC/IEEE 12207:2026**: Systems and software engineering – Software life cycle processes. Genebra: ISO, 2026. Disponível em: [CONFERIR URL]. Acesso em: 26 out. 2026.

ISO/IEC/IEEE. **ISO/IEC/IEEE 29148:2018**: Systems and software engineering – Life cycle processes – Requirements engineering. 2. ed. Genebra: ISO, 2018. Disponível em: https://www.iso.org/standard/72089.html. Acesso em: 26 out. 2026.

OWASP. **Authentication Cheat Sheet**. OWASP Cheat Sheet Series, s.d.b. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. Acesso em: 26 out. 2026.

OWASP. **Password Storage Cheat Sheet**. OWASP Cheat Sheet Series, s.d.c. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: 26 out. 2026.

SOMMERVILLE, I. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2018 [CONFERIR ano da edição brasileira].

TEMOSHOK, D. et al. **NIST SP 800-63B-4**: Digital Identity Guidelines: Authentication and Authenticator Management. Gaithersburg: National Institute of Standards and Technology, jul. 2025. DOI: https://doi.org/10.6028/NIST.SP.800-63B-4. Disponível em: https://csrc.nist.gov/pubs/sp/800/63/b/4/final. Acesso em: 26 out. 2026.

ANTHROPIC. Claude. 2026e. Disponível em: https://claude.ai. Acesso em: 26 out. 2026. Prompt: Apêndice VI, Seção VI.6 [conferir número da seção].

MICROSOFT. Common web application architectures. Microsoft Learn, 2023. Disponível em: https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures. Acesso em: 26 out. 2026.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2018. (já está na lista; acrescentar a página na citação)

SPRING. Spring Framework Reference: Controller Advice. s.d.i. Disponível em: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-advice.html. Acesso em: 26 out. 2026.

SPRING. Spring Framework Reference: JDBC Core Classes (JdbcTemplate). s.d.j. Disponível em: https://docs.spring.io/spring-framework/reference/data-access/jdbc/core.html. Acesso em: 26 out. 2026.

BRAY, T. (ed.). RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format. IETF, dez. 2017. Disponível em: https://www.rfc-editor.org/rfc/rfc8259.html. Acesso em: 25 out. 2026.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. (ed.). RFC 9110: HTTP Semantics. IETF, jun. 2022a. Disponível em: https://www.rfc-editor.org/rfc/rfc9110.html. Acesso em: 24 out. 2026.

OWASP. Authentication Cheat Sheet. OWASP Cheat Sheet Series, s.d.b. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. Acesso em: 26 out. 2026.

OWASP. Password Storage Cheat Sheet. OWASP Cheat Sheet Series, s.d.c. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: 26 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation: Unique Indexes. s.d.f. Disponível em: https://www.postgresql.org/docs/current/indexes-unique.html. Acesso em: 27 out. 2026. (nova)

SOMMERVILLE, I. Engenharia de software. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

SPRING. Spring Framework Reference: @RequestBody. s.d.d. Disponível em: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-methods/requestbody.html. Acesso em: 25 out. 2026.

SPRING. Spring Framework Reference: Exceptions. s.d.g. Disponível em: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-servlet/exceptionhandlers.html. Acesso em: 26 out. 2026.

SPRING. Spring Framework Reference: Controller Advice. s.d.i. Disponível em: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-advice.html. Acesso em: 26 out. 2026.

SPRING. Spring Framework Reference: JDBC Core Classes (JdbcTemplate). s.d.j. Disponível em: https://docs.spring.io/spring-framework/reference/data-access/jdbc/core.html. Acesso em: 26 out. 2026.

SPRING. Spring Security Reference: Password Storage. s.d.k. Disponível em: https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html. Acesso em: 27 out. 2026. (nova)

TEMOSHOK, D. et al. NIST SP 800-63B-4: Digital Identity Guidelines: Authentication and Authenticator Management. Gaithersburg: National Institute of Standards and Technology, jul. 2025. DOI: https://doi.org/10.6028/NIST.SP.800-63B-4. Disponível em: https://csrc.nist.gov/pubs/sp/800/63/b/4/final. Acesso em: 26 out. 2026.

Apêndice I – Códigos PlantUML

Figura 1 – código PlantUML do Diagrama de implantação do ambiente Web

```
@startuml
skinparam backgroundColor white
skinparam shadowing false
skinparam dpi 150
left to right direction
skinparam defaultFontName Arial
actor "Usuário" as U
node "Máquina do usuário" as M {
component "Navegador" as NAV
}
node "Servidor DNS" as DNS
node "Servidor da aplicação" as S {
component "Servidor web\n(Nginx)" as WEB
component "Servidor de aplicação\n(Spring Boot com Tomcat)" as APP
}
database "SGBD\n(PostgreSQL)" as DB
U --> NAV : digita o endereço\nou envia o formulário
NAV ..> DNS : 1. descobre o IP do site
NAV --> WEB : 2. HTTPS (TLS)\nporta 443
WEB --> APP : 3. HTTP interno\nporta 8080
APP --> DB : 4. SQL por JDBC\nporta 5432
@enduml
```

Figura 2 – código PlantUML do Diagrama de sequência geral de uma requisição Web

```
@startuml
skinparam backgroundColor white
skinparam shadowing false
skinparam dpi 150
skinparam defaultFontName Arial
skinparam sequenceMessageAlign center
actor "Usuário" as U
participant "Navegador" as N
participant "Servidor DNS" as D
```

```
participant "Servidor web\n(Nginx)" as W
participant "Servidor de aplicação\n(Spring Boot com Tomcat)" as A
database "SGBD\n(PostgreSQL)" as B
U -> N : digita o endereço ou envia o formulário
N -> D : pergunta o IP do site
D --> N : devolve o IP
N -> W : abre conexão TCP e TLS
N -> W : envia o pedido HTTP
W -> A : repassa o pedido
A -> B : consulta os dados
B --> A : devolve linhas ou erro
A --> W : monta a resposta
W --> N : repassa a resposta
N --> U : mostra o resultado na tela
@enduml
```

Figura 3 – código PlantUML do Diagrama de sequência da comunicação em rede na autenticação

```
@startuml figura3_sequencia_rede
participant "Navegador" as NAV
participant "Servidor DNS" as DNS
participant "Servidor web\n(Nginx)" as NGINX
participant "Servidor de aplicação\n(Spring Boot com Tomcat)" as APP
== DNS ==
NAV -> DNS : consulta o IP do nome do site (porta 53)
note right of DNS
O resolvedor responde pelo cache
ou consulta outros servidores DNS
end note
DNS --> NAV : devolve o endereço IP
== TCP (three-way handshake) ==
NAV -> NGINX : SYN (porta 443)
NGINX --> NAV : SYN + ACK
NAV -> NGINX : ACK
note over NAV, NGINX : canal confiável e ordenado aberto
== TLS 1.3 (handshake simplificado) ==
NAV -> NGINX : ClientHello (versões, cifras, parte da troca de chaves)
NGINX --> NAV : ServerHello + certificado + assinatura + Finished
note left of NAV
Verifica o certificado e
deriva as chaves de sessão
end note
NAV -> NGINX : Finished
note over NAV, NGINX : canal cifrado e autenticado
== HTTP (dentro do canal TLS) ==
NAV -> NGINX : POST /api/autenticacao\n{"email", "senha"}
NGINX -> APP : repassa o pedido\n(HTTP interno, porta 8080; ver Item
1.3)
note over APP : valida as credenciais no SGBD\n(detalhado no Item 1.3)
alt credenciais válidas
APP --> NGINX : 200 OK\n{"id", "email", "nome"}
else credenciais inválidas ou usuário inexistente
APP --> NGINX : 401 Unauthorized\nWWW-Authenticate:
Credenciais\n{"mensagem": genérica}
end
NGINX --> NAV : repassa a resposta (200 ou 401)
@enduml
```

Figura 4 – código PlantUML do Diagrama de sequência do atendimento de um recurso estático pelo Nginx

```
@startuml figura4_recurso_estatico
skinparam backgroundColor white
skinparam shadowing false
skinparam dpi 150
skinparam defaultFontName Arial
skinparam sequenceMessageAlign center
participant "Navegador" as NAV
participant "Servidor web\n(Nginx)" as NGINX
database "Disco do servidor\n(/var/www/aplicacao)" as DISCO

== Porta 80: redirecionamento ==
NAV -> NGINX : GET http://aplicacao.example/ (TCP, porta 80)
NGINX --> NAV : 301 Moved Permanently\nLocation:
https://aplicacao.example/

== Porta 443: recurso estático (dentro do canal TLS) ==
NAV -> NGINX : GET / (TCP com TLS, porta 443)
note right of NGINX
escolhe o bloco location /
e junta o caminho ao root
end note
NGINX -> DISCO : lê o arquivo
alt arquivo existe
  DISCO --> NGINX : conteúdo do arquivo
  NGINX --> NAV : 200 OK + HTML, CSS ou JavaScript\n+
Strict-Transport-Security
else arquivo não existe
  NGINX --> NAV : 404 Not Found
end
@enduml
```

Figura 5 – código PlantUML do Diagrama de sequência do processamento da autenticação no lado servidor

```
@startuml figura5_recurso_dinamico
skinparam backgroundColor white
skinparam shadowing false
skinparam dpi 150
skinparam defaultFontName Arial
skinparam sequenceMessageAlign center
participant "Navegador" as NAV
participant "Servidor web\n(Nginx)" as NGINX
box "Servidor de aplicação" #FFFFFF
participant "Tomcat\n(conector)" as CON
participant "Dispatcher\nServlet" as DS
participant "Código da\naplicação" as APP
end box
database "SGBD\n(PostgreSQL)" as BD

NAV -> NGINX : POST /api/autenticacao\n{"email", "senha"}
NGINX -> CON : repassa (HTTP, 127.0.0.1:8080)\n+ X-Forwarded-For e
-Proto
CON -> DS : thread entrega\nrequest e response
DS -> DS : converte o JSON\ne valida os campos
alt JSON malformado ou campos inválidos
  DS --> CON : 400 + JSON de erro
else dados válidos
```

```
    DS -> APP : invoca o método\nde autenticação
    APP -> BD : SELECT ... WHERE email = ?\n(JDBC, pool, porta 5432)
    note right of BD
    backend: análise, reescrita,
    planejamento e execução
    end note
    alt consulta executada
      BD --> APP : 1 linha ou 0 linhas
      APP --> DS : 200 (dados do usuário)\nou 401 (WWW-Authenticate
  e\nmensagem genérica)
    else falha no SGBD
      BD --> APP : erro com código SQLSTATE
      APP --> DS : exceção tratada pelo\n@RestControllerAdvice: 500
    end
    DS --> CON : resposta convertida em JSON
  end
  CON --> NGINX : resposta HTTP
  NGINX --> NAV : resposta no canal TLS
  note over NAV, NGINX
  se o Tomcat estiver parado ou não
  responder a tempo: 502 ou 504
  end note
  @enduml
```

Figura 6 – código PlantUML do Diagrama de caso de uso da aplicação de autenticação

```
@startuml figura6_caso_de_uso
skinparam backgroundColor white
skinparam shadowing false
skinparam dpi 150
skinparam defaultFontName Arial
left to right direction

actor "Usuário" as U

rectangle "Aplicação Web de autenticação" {
  usecase "Autenticar usuário\n(RF01)" as UC1
}

U -- UC1
@enduml
```

Figura 7 – código PlantUML do Diagrama de componentes da aplicação de autenticação

```
@startuml figura7_componentes
   skinparam backgroundColor white
   skinparam shadowing false
   skinparam dpi 150
   skinparam defaultFontName Arial
   skinparam componentStyle rectangle
   skinparam packageStyle rectangle

   node "Máquina do usuário (navegador)" as M {
     package "Camada 1: Apresentação" as C1 {
```

```
        component "Página de login\n(HTML, CSS e JavaScript)" as PAG
      }
    }

    node "Servidor da aplicação" as S {
      component "Servidor web\n(Nginx)" as NGX
      node "Servidor de aplicação\n(Spring Boot com Tomcat)" as APP {
        package "Camada 2: Controle" as C2 {
          component "TratadorExcecoes\n(@RestControllerAdvice)" as TRT
          component "ControladorAutenticacao" as CTL
        }
        package "Camada 3: Serviço de autenticação" as C3 {
          component "ServicoAutenticacao" as SRV
          component "VerificadorSenha" as VER
        }
        package "Camada 4: Persistência" as C4 {
          component "RepositorioUsuario\n(Spring JDBC)" as REP
        }
      }
    }

    database "SGBD\n(PostgreSQL)" as DB

    PAG -down-> NGX : POST /api/autenticacao (JSON)\nHTTPS, porta 443
    NGX .up.> PAG : entrega os arquivos da página
    NGX -[hidden]right-> APP
    NGX --> CTL : repassa o pedido\nHTTP interno, porta 8080
    TRT -[hidden]down-> CTL
    CTL .up.> TRT : exceções lançadas pelo\ncontrole e pelo serviço
    CTL -down-> SRV : autentica as credenciais
    SRV -right-> VER : confere a senha\ncom o hash
    SRV -down-> REP : busca o usuário\npelo e-mail
    REP -down-> DB : SQL parametrizado\nJDBC, porta 5432
    @enduml
```

Figura 8 – código PlantUML do Diagrama de classes da aplicação de autenticação

```
@startuml figura8_classes
skinparam backgroundColor white
skinparam shadowing false
skinparam dpi 150
skinparam defaultFontName Arial
skinparam defaultFontSize 15
skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
hide empty members

package "Camada 2: Controle" {
  class ControladorAutenticacao <<RestController>> {
    - servico : ServicoAutenticacao
    + autenticar(requisicao) : UsuarioAutenticado
  }
  class TratadorExcecoes <<RestControllerAdvice>> {
    + jsonInvalido(e) : 400
    + validacao(e) : 400
    + falhaAutenticacao(e) : 401
    + erroInesperado(e) : 4xx do Spring ou 500
  }
}

package "Camada 3: Serviço de autenticação" {
  class ServicoAutenticacao <<Service>> {
    - repositorio : RepositorioUsuario
    - verificador : VerificadorSenha
    + autenticar(email, senha) : UsuarioAutenticado
```

```
  }
  class VerificadorSenha <<Component>> {
    - codificador : PasswordEncoder
    - hashFicticio : String
    + confere(senha, hashSenha) : boolean
    + verificarContraHashFicticio(senha) : void
  }
  class ConfiguracaoSenha <<Configuration>> {
    + codificadorSenha() : PasswordEncoder
  }
  class FalhaAutenticacaoException <<exception>>
}

package "Camada 4: Persistência" {
  class RepositorioUsuario <<Repository>> {
    - jdbcTemplate : JdbcTemplate
    + buscarPorEmail(email) : Optional<Usuario>
  }
}

package "Modelo" {
  class RequisicaoAutenticacao <<record>> {
    email : String
    senha : String
  }
  class UsuarioAutenticado <<record>> {
    id : long
    email : String
    nome : String
  }
  class Usuario <<record>> {
    id : long
    email : String
    nome : String
    hashSenha : String
  }
  class RespostaErro <<record>> {
    mensagem : String
  }
}

ControladorAutenticacao -down-> ServicoAutenticacao : autenticar
ServicoAutenticacao -down-> RepositorioUsuario : buscarPorEmail
ServicoAutenticacao -right-> VerificadorSenha : confere
ConfiguracaoSenha .up.> VerificadorSenha : fornece o\nPasswordEncoder
ServicoAutenticacao ..> FalhaAutenticacaoException : lança
TratadorExcecoes ..> FalhaAutenticacaoException : trata
RepositorioUsuario -[hidden]down-> RequisicaoAutenticacao
@enduml
```

Figura 9 – código PlantUML do Diagrama de sequência da autenticação: sucesso, senha incorreta e usuário inexistente

```
@startuml figura9_autenticacao
skinparam backgroundColor white
skinparam shadowing false
skinparam dpi 150
skinparam defaultFontName Arial
skinparam sequenceMessageAlign center
participant "ControladorAutenticacao" as CTL
participant "ServicoAutenticacao" as SRV
participant "RepositorioUsuario" as REP
participant "VerificadorSenha" as VER
participant "TratadorExcecoes" as TRT
database "SGBD\n(PostgreSQL)" as BD
```

```
[-> CTL : POST /api/autenticacao\n{"email", "senha"} (válido)
CTL -> SRV : autenticar(email, senha)
SRV -> REP : buscarPorEmail(email)
REP -> BD : SELECT id, email, nome, hash_senha\nFROM usuario WHERE email = ?
alt usuário existe
  BD --> REP : 1 linha
  REP --> SRV : Optional com Usuario
  SRV -> VER : confere(senha, hashSenha)
  alt senha confere
    VER --> SRV : true
    SRV --> CTL : UsuarioAutenticado(id, email, nome)
    [<-- CTL : 200 OK\n{"id", "email", "nome"}
  else senha incorreta
    VER --> SRV : false
    SRV ->> TRT : lança FalhaAutenticacaoException
    [<-- TRT : 401 Unauthorized\nWWW-Authenticate: Credenciais
realm="aplicacao.example"\n{"mensagem": "E-mail ou senha inválidos."}
  end
else usuário inexistente
  BD --> REP : 0 linhas
  REP --> SRV : Optional vazio
  SRV -> VER : verificarContraHashFicticio(senha)
  note right of VER
  mesmo custo de uma
  verificação real (RNF06);
  o resultado é descartado
  end note
  VER --> SRV
  SRV ->> TRT : lança FalhaAutenticacaoException
  [<-- TRT : 401 Unauthorized\nWWW-Authenticate: Credenciais
realm="aplicacao.example"\n{"mensagem": "E-mail ou senha inválidos."}
end
@enduml
```

Figura 10 – código PlantUML do Diagrama de sequência dos fluxos de erro: dados malformados e falha no SGBD

```
@startuml figura10_erros
skinparam backgroundColor white
skinparam shadowing false
skinparam dpi 150
skinparam defaultFontName Arial
skinparam sequenceMessageAlign center
participant "DispatcherServlet\n(Spring MVC)" as DS
participant "ControladorAutenticacao" as CTL
participant "ServicoAutenticacao" as SRV
participant "RepositorioUsuario" as REP
participant "TratadorExcecoes" as TRT
database "SGBD\n(PostgreSQL)" as BD

[-> DS : POST /api/autenticacao
alt JSON malformado ou campo ausente, vazio ou com e-mail inválido (FA1)
  DS -> DS : converte o JSON e valida\n@NotBlank e @Email
  DS ->> TRT : HttpMessageNotReadableException\nou MethodArgumentNotValidException
  [<-- TRT : 400 Bad Request\n{"mensagem": "Requisição inválida."}
  note right of TRT
  o controlador e o SGBD
  não são chamados
  end note
else SGBD indisponível ou com erro (FE1)
  DS -> CTL : autenticar(requisicao)
  CTL -> SRV : autenticar(email, senha)
  SRV -> REP : buscarPorEmail(email)
  REP -> BD : SELECT ... WHERE email = ?
  BD --> REP : erro (SQLSTATE)
  REP ->> TRT : DataAccessException
```

```
  TRT -> TRT : registra o erro\nsó no log do servidor
  [<-- TRT : 500 Internal Server Error\n{"mensagem": "Erro interno. Tente novamente
mais tarde."}
end
@enduml
```

Apêndice II – Atas e Comunicações

Este apêndice registra os combinados da equipe, as reuniões e as comunicações entres os membros.

II.1 – Combinados da equipe

Os combinados abaixo foram definidos por consenso no início do trabalho, e valem para todo o projeto.

Tabela II.1 – Combinados da equipe

Combinado	**Definição**
Canal oficial da equipe	Servidor no Discord chamado “
Encontros síncronos	Tivemos um encontro síncrono no dia 26/10
Tarefas assíncronas	No mínimo 1 encontró sincróno
Documento compartilhado	Google Docs com histórico de versões: https://docs.google.com/document/d/1h6EnYMmzYm5wL05_vyfGRqvagQijY1W9SVC_ZTv2-B4/edit?usp=sharing
Gerência de configuração	Repositório GitHub isadora-yasmim/aps-dominios-de-software (criado em 19/10/2026)
Pilha de referência	Navegador → Nginx → Spring Boot com Tomcat → PostgreSQL
Padrões	Diagramas em PlantUML; referências pela ABNT NBR 6023; uso de IA citado conforme a Regra 8
Matriz de autoria e revisão	Conforme a tabela de autoria e revisão do início do documento

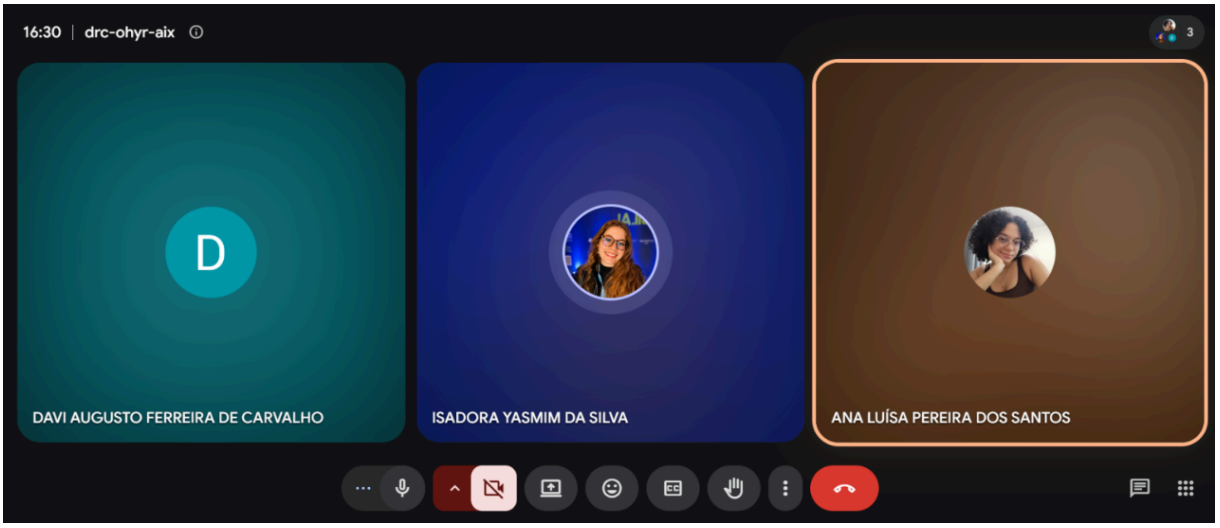
II.2 – Atas das reuniões

Cada reunião síncrona tem uma ata, numerada em ordem. A linha “Situação dos compromissos anteriores” mostra o acompanhamento contínuo das tarefas de cada membro, e a linha “Riscos revistos” liga a reunião ao Apêndice IV. Copie a tabela abaixo para cada reunião.

Tabela II.2 – Ata 01

Campo	Registro
Número, data, horário e meio	01 – [26/10/2026] – [16h00 às 16h30] – [Google Meet]
Presentes	Ana Luísa, Davi Augusto, Isadora Yasmin
Ausentes	Ninguém
Decisões	Revisão do plano guia, qual stack seria utilizada no desenvolvimento, como ficaria a formatação do documento e como seriam redigidas as referências.
Redigida por / validada por	[Davi Augusto] / [Ana Luísa]

O anexo a seguir é um print, realizado pela Ana Luísa, após o término da reunião, às 16:30:



II.3 – Registro de comunicações da equipe

A Tabela II.3 lista, em ordem cronológica, as comunicações que registram decisões, combinados, entregas, cobranças e atrasos. Conversas sociais ficam de fora. As evidências (prints ou exportações do canal) estão no repositório e no canal oficial.

Tabela II.3 – Registro de comunicações da equipe

Data	Canal	Participantes	Assunto ou decisão	Evidência
13/10/2026	Aula da disciplina	Todos	*Kickoff* da APS pela docente	Definição da APS
19/10/2026	GitHub	Isadora	Criação do repositório aps-dominios-de-software	Primeiro commit do repositório
21/10/2026	Discord	Todos	Plano de trabalho e matriz de autoria e revisão, elaborados com apoio de IA	docs/apendices/I-plano/Guia-da-APS-trio.pdf

Data	Canal	Participantes	Assunto ou decisão	Evidência
24/10/2026	Google Docs	Ana	Item 1.1 em rascunho	Histórico de versões
25/10/2026	Google Docs	Isadora	Item 1.2 em rascunho	Histórico de versões
25/10/2026	Google Docs	Davi	Item 1.3 em rascunho, com observações sobre os Itens 1.1 e 1.2	Histórico de versões
26/10/2026	Discord	Isadora, Davi	Revisão do Item 1.3 entregue e correções aplicadas pelo autor	docs/apendices/V-revisoes/Revisao-Item-1.3.pdf
26/10/2026	Google Docs	Isadora; Ana	Itens 2.1 e 2.2 em rascunho	Histórico de versões
27/10/2026	GitHub	Isadora	Implementação da autenticação enviada à branch feature/autenticacao	Commits da branch
27/10/2026	Discord	Davi, Ana	Revisão do Item 2.2 entregue à autora	docs/apendices/V-revisoes/Revisao-Item-2.2.docx
27/10/2026	Discord	Davi	Item 2.3 escrito a partir do código da feature/autenticacao	Histórico de versões
27/10/2026	Discord	Isadora, Ana	Revisão do Item 2.3 entregue ao autor	docs/apendices/V-revisoes/Revisao-Item-2.3.pdf

Apêndice III – Prompts de IA